

XIRASM 中文语言指南

自然 ISA 文本与现代编译期语言

版本 0.2.16

生成日期 2026-07-19

目录

XIRASM 语言指南	7
指南结构	7
第一部分：语言基础	7
第二部分：汇编器模型	7
第三部分：构建程序	8
第 1 章：认识 XIRASM	9
同一份源文件，两种写法	9
运行一个例子	9
编译期代码生成输出	9
源文件语法注意点	10
怎么读 XIRASM 源码	10
第 2 章：值与绑定	12
值只在编译期存在	12
常量	12
可变绑定	12
类型注解和类型推断	13
字符串和字节序列	13
块作用域	14
选择 const 还是 let	14
第 3 章：表达式	15
表达式产生编译期值	15
算术运算	15
位运算和移位	15
比较和相等判断	16
布尔逻辑和短路求值	16
表达式中的函数调用	17
字段访问	17
运算符优先级	17
表达式错误	18
第 4 章：控制流	19
控制流在编译期执行	19
if 和 else	19
选择 ISA 指令	20
目标条件	20
遍历范围	20
遍历列表	21

while	21
break 和 continue	21
选择控制流形式	21
第 5 章：函数与作用域	23
函数封装编译期工作	23
过程函数	23
参数和实参	23
返回值函数	24
返回规则和副作用	24
函数局部作用域	25
声明顺序	25
递归和调用深度	26
选哪种函数	26
第 6 章：集合与文本	27
集合是编译期值	27
字符串表示源文本	27
分割和连接文本	27
用字节序列表示二进制值	28
构建字节序列	28
列表保持顺序	28
更新 `let` 绑定的集合	29
列表可以保存更大的值	29
映射保存命名值	29
遍历映射内容	30
在函数中组合集合	30
选哪种集合	30
第 7 章：词法单元与模式匹配	32
词法单元表示源代码结构	32
匹配词法单元结构	32
字面量模式词法单元	33
捕获种类	33
平衡的词法单元范围	34
最小匹配与回溯	34
空词法单元范围	34
匹配已有词法单元列表	35
尝试不同结构	35
匹配失败与无效模式	35

字符串还是词法单元	36
第 8 章：目标、ISA 文本与标号	37
当前目标	37
选择 x86、RISC-V 和 SPIR-V	37
查询目标	38
自然 ISA 文本	38
在指令中使用编译期值	39
静态标号	39
符号引用与地址回填	40
动态指令文本与动态标号	40
读取标号地址	40
实用规则	41
第 9 章：数据与二进制布局	42
输出内容是有序字节序列	42
浮点值	42
字符串与字节序列	43
预留空间	43
填充与对齐	44
声明二进制结构体	44
字段默认值与结构体字面量	45
测量布局	45
在 x86-64 栈上使用结构体布局	46
打包并写出结构体值	46
联合体与当前字段	47
选择布局方式	48
第 10 章：模块与文件	49
导入源模块	49
内联包含源文件	49
源文件路径解析	50
选择 `import` 还是 `include`	50
检查与读取数据文件	50
读取字节范围	51
加载 JSON	51
加载 TOML	52
区分配置与二进制数据	52
实用的模块组织方式	52
第 11 章：输出区域与虚拟数据	54

逻辑坐标与物理坐标	54
设置 Origin	54
创建显式区域	55
实际光标与潜在光标	55
裁掉预留尾部	56
保留潜在间隙	56
对齐区域文件大小	56
虚拟输出	57
使用当前地址作为虚拟 Origin	57
稳定的区域事实	58
选择合适的工具	58
第 12 章：收尾处理	60
为回填预留字段	60
读取最终映像	61
计算校验和	61
使用稳定的区域信息	62
调用值函数	62
可复用的回填过程	63
收尾处理的控制流	63
收尾处理的执行顺序	63
必须延后运行的布局工作	64
后期布局可物理化预留尾部	65
组合后期布局与最终回填	65
后期布局的限制	66
选择正确的阶段	66
第三部分：构建程序	66
第 13 章：Flat Binary 与自定义文件格式	68
原始机器码	68
组合文件头与 Payload	68
用 Struct 描述记录	68
推导文件头字段	69
区分逻辑地址与文件偏移	70
表示已初始化间隙和 File-Free 尾部	70
Late Layout：追加尾部或放置晚生成表	71
断言确认文件正确	71
自定义格式的组织顺序	72
何时使用格式接口	72

第 14 章：可执行文件与目标文件格式	73
先声明映像再写内容	73
完整示例：ELF64 程序	73
Sections 与 Segments	74
格式族	74
入口点绑定	74
生成的格式内容	75
常规接口与高级接口	75
后续阅读	75
第 15 章：诊断与实用惯例	77
阅读诊断信息	77
信息与警告	77
主动报错	77
断言	77
保护目标相关源码	78
用名字替代魔数	79
在名称中写清坐标	79
推导布局事实	79
保持收尾处理聚焦	79
按职责分模块	80
选择正确的格式层	80
在输出边界检查	80
找到合适的文档	80
最终检查清单	81

XIRASM 语言指南

XIRASM 是一款面向 x86、RISC-V 和 SPIR-V 的现代汇编器。ISA 指令保持熟悉的汇编写法；值、函数、控制流、集合、数据布局和输出格式则由结构化的编译期语言表达。

本指南按照实际编程顺序介绍 XIRASM：先学习值、表达式和函数，再进入标号、数据布局、输出区域与收尾处理，最后构建 flat binary 和标准可执行格式。需要查询精确签名时使用语言 API 参考；需要构建 PE、COFF 或 ELF 文件时使用格式教程。

指南结构

第一部分：语言基础

- 认识 XIRASM
- 汇编第一个 x86-64 程序。
- 理解 ISA 文本与编译期代码如何配合。
- 掌握基本源代码规则。
- 值与绑定
- 声明常量和可变的局部绑定。
- 使用整数、布尔值、字符串和字节。
- 理解作用域和赋值。
- 表达式
- 使用算术、比较、逻辑、位运算和字段表达式。
- 调用函数并组合编译期计算。
- 控制流
- 使用 `if` 选择需要生成的内容。
- 使用 `while` 和 `for` 重复执行操作。
- 根据编译期条件生成数据和指令。
- 函数与作用域
- 编写不返回值的过程和返回值的函数。
- 使用参数、返回类型、局部绑定和嵌套作用域。
- 集合与文本
- 创建和处理列表、映射、字符串及字节序列。
- 使用集合描述表格和需要生成的输出内容。
- 词法单元与模式匹配
- 检查词法单元序列。
- 匹配源代码模式，并据此构建更高层的源代码处理工具。

第二部分：汇编器模型

- 目标、ISA 文本与标号

- 选择指令集和指令模式。
- 定义标号，并在指令中使用编译期值。
- 理解地址引用与回填。
- 数据与二进制布局
- 写出整数、字符串和字节。
- 预留空间并控制对齐方式。
- 定义具有精确二进制布局的结构体和联合体。
- 模块与文件
- 通过包含和导入复用源代码。
- 从普通文件以及 JSON、TOML 格式中读取源数据。
- 输出区域与虚拟数据
- 区分逻辑地址、文件偏移和最终物理字节。
- 构建输出区域，表达预留空间、物理间隙和不占文件空间的尾部。
- 使用虚拟输出作为临时组装、测量和转换区域。
- 收尾处理
- 等布局稳定后再执行检查和回填。
- 计算校验和与最终字段值。
- 理解哪些操作不得改变最终布局。

第三部分：构建程序

- Flat Binary 与自定义格式
- 构建紧凑的 flat binary 和项目专用文件格式。
- 用标号、区域、`defer` 和 `late_layout` 推导文件头字段。
- 可执行文件与目标文件格式
- 理解普通格式接口负责完成哪些工作。
- 使用新版 `let / *_mut` 调用方式构建最小可执行文件。
- 通过格式教程继续学习 PE、COFF、ELF、导入、导出、BSS（未初始化数据区）和重定位。
- 诊断信息与实用约定
- 报告无效输入，并检查布局是否符合预期。
- 组织可以复用的汇编项目。
- 在掌握本指南后继续查阅接口参考和格式教程。

第 1 章：认识 XIRASM

同一份源文件，两种写法

XIRASM 源文件可以包含两种形式的代码：

- ISA 文本：描述所选处理器的指令，保持常规汇编写法。
- 编译期代码：在汇编过程中计算值、生成输出，并控制源码如何被组装。

它们属于同一种源语言。

编译期值可以直接出现在指令中，标号可以交给布局 API 使用，函数也可以同时生成数据和 ISA 文本。

例子：

```
// 开启 64 位 x86 模式。
x86.use64();

// 编译期计算常量，指令里直接用。
const answer: u32 = 40 + 2

entry:
// 把计算好的数值放进 eax，然后返回。
mov eax, answer
ret
```

简单说明：

- `x86.use64();` 选择 64 位 x86 指令编码模式。
- `const answer: u32 = 40 + 2` 声明编译期常量。
- `entry:` 在当前逻辑地址定义一个标号。
- `mov eax, answer` 是普通 x86 指令，把 `answer` 移入 `eax`，编译期自动代入常量值。
- `ret` 是普通 x86 指令。

`x86.use64();` 这类调用使用结构化的编译期语法。`mov eax, answer` 仍然是自然 ISA 文本，不需要写成函数调用。

运行一个例子

把源码保存为 `hello.xir`，然后执行：

```
xirasm hello.xir -o hello.bin --target x86-64
```

输出字节：

```
b8 2a 00 00 00 c3
```

前五个字节是 `mov eax, 42` 的编码，最后一个字节是 `ret`。标号不产生字节，只记录一个地址，供其他指令和编译期 API 使用。

直接汇编时，XIRASM 默认写出 flat binary，也就是源码实际生成的字节序列；它不带操作系统文件头。后面的章节会介绍格式接口，完整的 PE、COFF 和 ELF 程序见格式教程。

编译期代码生成输出

编译期代码在汇编过程中执行，不是在最终程序里运行。

比如循环写四个字节：

```
// 编译期循环四次，每次写一个字节。
const count: u8 = 4
for value in range(0, count) {
```

```
    db(value);
}
```

生成的字节：

```
00 01 02 03
```

`range(0, count)` 生成从 0 到 `count-1` 的序列。`for` 在编译期执行，每次调 `db(value)` 写一个字节。

这个区别很关键：

做什么	怎么写
手写 ISA 指令	ISA 文本
计算和判断	编译期表达式
重复或条件生成	编译期控制流
输出字节	<code>db</code> 、 <code>dw</code> 、 <code>dd</code> 、 <code>dq</code> 等数据调用
封装可复用逻辑	函数
生成 PE/COFF/ELF	对应的格式接口

源文件语法注意点

几条简单规则：

- `//` 到行尾是单行注释，也可以写在 ISA 指令后；ISA 字符串操作数中的 `//` 仍是普通文本。
- ISA 指令行不以分号结尾。
- 标号以 `:` 结尾。
- 函数和接口调用以 `;` 结尾。
- `return` 语句用 `;` 结尾。
- `const` 和 `let` 声明末尾不加分号。
- 代码块用 `{}`。
- 调用和返回值可以跨行，左括号会一直等到对应的右括号。

完整例子：

```
// 定义编译期可调用的辅助函数。
fn emit_marker(value: u8) {
    // 写固定标记，后面跟传入的值。
    db(0x7f, value);
}

const marker: u8 = 3
emit_marker(marker);

start:
// ISA 指令里可以直接使用编译期常量；指令行末尾不写分号。
mov eax, marker
ret
```

函数定义和函数调用属于编译期代码；标号和下面两行缩进的指令属于 ISA 文本。两种写法共同生成同一个最终输出。

怎么读 XIRASM 源码

读源码时，按这几步判断每行：

- 如果是 ISA 指令，就写自然 ISA 文本。

- 如果要在汇编期间计算、选择或重复生成内容，就使用编译期语言。
- 如果要写出字节或预留空间，就使用数据和输出 API。
- 如果要描述完整文件格式，就使用对应的格式接口，不要手算文件头表项。

下一章讲编译期语言基础：值、常量绑定和赋值。

[返回语言指南](#)

第 2 章：值与绑定

值只在编译期存在

XIRASM 的值都是编译期值。它们用来构建输出，但绑定本身不会写字节，也不会预留内存。

```
// 这里只创建编译期值，不会向输出写入任何字节。
const magic: u16 = 0x5a4d
```

这行把值赋给名字 **magic**，但不会把 **4d 5a** 写到输出里。要输出这个值，需要调用数据写出接口：

```
// 声明 16 位魔数，再通过数据接口按对应宽度写出。
const magic: u16 = 0x5a4d
dw(magic);
```

同样的规则适用于可变值、字符串、字节序列、集合以及函数返回值。只有指令、数据调用、布局操作或格式接口使用了这些值，它们才会成为输出的一部分。

常量

常量把一个名字绑定到一个不可能重新赋值的值：

```
// 页面大小和文件头大小在整个汇编过程中保持不变。
const page_size: u64 = 4096
const header_size = 64
// 布尔常量可直接用于后续的编译期条件。
const enabled: bool = true
```

一般形式是：

```
const name = expression
const name: type = expression
```

如果 XIRASM 能从初始值推断出类型，类型注解可以省略。当宽度或值类型属于二进制约定的的一部分时，应该写类型。

常量声明末尾不加分号。给常量重新赋值会报错：

```
const value = 1
value = 2
```

除非源文件在汇编时确实需要更新绑定，否则优先用 **const**。

可变绑定

编译期计算需要状态时用 **let**：

```
// 逐步增加偏移量，最后写出计算结果。
let offset: u32 = 0
offset = offset + 16
offset = offset + 4

dd(offset);
```

输出：

```
14 00 00 00
```

声明和赋值都在汇编时发生，不会在最终程序中创建名为 **offset** 的运行时变量。

形式是：

```
let name = expression
let name: type = expression
name = expression
```

和常量一样，赋值末尾不加分号。作为语句的调用仍然需要分号：

```
// 更新编译期绑定，然后把最终值写成一个字节。
let value = 1
value = value + 1
db(value);
```

类型注解和类型推断

绑定可以显式声明类型：

```
// 明确的类型让字段宽度和文本用途直接体现在源代码中。
const signature: u16 = 0x5a4d
const title: string = "XIRASM"
const marker: bytes = b"OK"
const enabled: bool = true
```

也可以靠推断：

```
// 这些值的类型都可以由字面量直接确定。
const count = 4
const name = "payload"
const raw = b"DATA"
```

常用的值类型：

类型	用途	示例
integer	通用编译期整数	42
u8	8 位无符号值	0xff
u16	16 位无符号值	0x5a4d
u32	32 位无符号值	0x401000
u64	64 位无符号值	0x140000000
i8	8 位有符号值	-1
i16	16 位有符号值	-200
i32	32 位有符号值	-4096
i64	64 位有符号值	-0x100000000
f32	IEEE-754 32 位浮点数	f32(1.5)
f64	IEEE-754 64 位浮点数	1.5
bool	编译期条件	true
string	编译期文本	"kernel"
bytes	确切的字节序列	b"PE"

列表、映射、结构体、联合体和类型值在后面的章节介绍。

指定位宽的整数类型适合表示二进制字段和函数接口。如果一个函数只关心值是整数而不在意具体宽度，可以用 **integer** 类型。最终输出时写多少个字节由调用接口决定。

有符号固定位宽整数使用补码表示。创建绑定或结构字段时会检查声明范围；打包有符号字段时，会按字段宽度写出低 8、16、32 或 64 位。

带小数部分或指数的十进制字面量类型为 **f64**。用 **f32(value)** 显式窄化为 **f32**，用 **f64(value)** 把 **f32** 扩展为 **f64**。整数和浮点值之间不会隐式转换。

字符串和字节序列

字符串和字节序列是不同类型的值：

```
// 节名是文本，而文件签名表示精确的两个字节。
const section_name: string = ".text"
const signature: bytes = b"MZ"
```

文本名称、路径、生成的指令文本以及需要文本的接口，用字符串。需要精确字节序列时用 **bytes**。

有些输出接口两种类型都接受：

```
// 字符串和显式字节序列会按参数顺序连续写出。
db("AB", b"CD");
```

输出四个字节：

```
41 42 43 44
```

其他接口只接受其中一种。在绑定中区分清楚，能让这些约定在源码里更明显。

块作用域

绑定属于它声明时所在的作用域。块会创建一个嵌套作用域：

```
// 外层常量在内部代码块结束后仍然可见。
const value = 1

{
  // 内层绑定只在这个代码块中遮蔽外层同名常量。
  let value = 2
  // 离开代码块后，再次使用外层的 value。
  db(value);
}

db(value);
```

输出：

```
02 01
```

内部的 **value** 只在块内遮蔽外部的常量。块结束后，外部的 **value** 重新可见。

函数参数和局部绑定也属于函数作用域。嵌套块可以引入临时名字，不影响外部绑定。

当嵌套操作确实需要一个自然的局部名字时，遮蔽是有用的。但如果两个含义放在源码里很难区分，就不要用相同的名字。

选择 **const** 还是 **let**

选最精确的绑定来表达计算：

情况	优先选
固定选项、大小、名称或计算结果	const
语言自身管理的循环计数器	循环绑定
累计的偏移量、校验和或累加器	let
只在小范围代码块里变化的值	块内 let
宽度属于文件格式约定的值	显式类型注解
类型显而易见的临时值	类型推断

格式描述和汇编生成逻辑中的大多数绑定应该是常量。可变状态在局部、短生命周期、只跟一个计算相关时最清晰。

下一章讲产生这些值的表达式：算术、比较、布尔逻辑、位运算、字段访问和函数调用。

返回语言指南

第 3 章：表达式

表达式产生编译期值

表达式在 XIRASM 汇编时计算值。表达式可以用在声明、赋值、函数参数、return 语句、条件、断言、指令操作数和数据调用中。

```
// 根据表项数量和单项大小，在汇编期间计算整张表的字节数。
const entry_count = 3
const bytes_per_entry = 8
const table_size = entry_count * bytes_per_entry

dd(table_size);
```

输出 32 位值 **24**。乘法在汇编时执行，输出只有计算结果。

算术运算

整数表达式支持：

运算符	含义
+	加法
-	减法
*	乘法
/	整数除法
%	取余

```
// 同时演示默认优先级、括号分组、整数除法和取余。
const total = 2 + 3 * 4
const grouped = (2 + 3) * 4
const quotient = 17 / 5
const remainder = 17 % 5

dd(total, grouped, quotient, remainder);
```

输出值分别是 **14**、**20**、**3**、**2**。

加、减、乘会检查溢出。结果超出支持的整数范围时报错，不会静默截断。除数为零会报错。

一元 **+** 保持值不变。一元 **-** 产生 64 位补码值：

```
// -1 的 64 位二进制补码中，每一位都为 1。
const all_bits = -1
dq(all_bits);
```

输出八个 **ff** 字节。

位运算和移位

位运算用于权限、掩码、指令字段和文件格式标志：

运算符	含义	
&	按位与	
\	,	按位或
^	按位异或	
~	按位取反	
<<	左移	
>>	右移	

```
// 每一位分别表示一种权限。
```

```

const readable = 1 << 0
const writeable = 1 << 1
const executable = 1 << 2

// 组合读取与执行权限，再分别检查两个权限位。
const permissions = readable | executable
const can_execute = (permissions & executable) != 0
const can_write = (permissions & writeable) != 0

assert(can_execute);
assert(!can_write);
db(permissions);

```

输出字节是 **05**。

掩码测试用括号包起来，这样意图更清楚，也不依赖位运算和比较运算的优先级。

比较和相等判断

表达式可以比较值：

运算符	含义
<code>==</code>	等于
<code>!=</code>	不等于
<code><</code>	小于
<code><=</code>	小于等于
<code>></code>	大于
<code>>=</code>	大于等于

大小比较作用于整数：

```

// 数据必须大于零，并且不能超过允许的最大大小。
const payload_size = 96
const maximum_size = 128
const fits = payload_size > 0 && payload_size <= maximum_size

assert(fits);

```

相等和不相等也可以用于兼容的非整数类型：

```

// 分别核对字符串名称和精确的字节签名。
const format_name = "raw"
const signature = b"OK"

assert(format_name == "raw");
assert(signature == b"OK");

```

不相关的值类型之间比较会报错，不会隐式转换。

布尔逻辑和短路求值

布尔逻辑支持：

运算符	含义		
<code>!</code>	逻辑非		
<code>&&</code>	逻辑与		
<code>\</code>	<code>\</code>	<code>,</code>	逻辑或

`&&` 和 `||` 会短路：

- `left && right` 当 `left` 为 `false` 时不计算 `right`。
- `left || right` 当 `left` 为 `true` 时不计算 `right`。


```
// 左侧已经为真，因此右侧包含除零的表达式不会被计算。
const enabled = true
const safe = enabled || (1 / 0 == 0)

assert(safe);
```

除法不会被执行，因为 **enabled** 是 **true**。

短路求值在后面的表达式需要前面的条件成立时很有用。

表达式中的函数调用

返回值函数或内置函数可以出现在任何能接受其结果类型的地方：

```
// 计算名称长度，并把 37 向上调整到 16 的整数倍。
const name_length = lengthof("XIRASM")
const aligned_size = ((37 + 15) / 16) * 16

db(name_length);
dd(aligned_size);
```

调用可以嵌套：

```
// 先去掉两端空格，再把结果转换为大写。
const normalized = upper(trim(" kernel "))
assert(normalized == "KERNEL");
```

只做输出或汇编动作的过程函数以语句方式调用，不产生表达式值。返回值函数在第 5 章介绍。

字段访问

有命名字段的值用 **.** 选择一个字段：

```
header.magic
header.entry_offset
```

结构体和联合体值在第 9 章介绍。**target.bits** 和 **target.isa** 这类目标查询属于目标条件，应该直接写在 **if** 条件里，不要先复制到普通绑定中。

运算符优先级

表中同一行的运算符优先级相同。越靠上的行绑定越紧密：

优先级	运算符		
最高	函数调用、括号表达式、字段访问		
一元	+ 、 - 、 ~ 、 !		
乘法	* 、 / 、 %		
移位	<< 、 >>		
加法	+ 、 -		
按位与	&		
按位异或	^		
按位或	 	,	
大小比较	< 、 <= 、 > 、 >=		
相等判断	== 、 !=		
逻辑与	&&		
最低	 	 	,

同优先级从左到右计算。

XIRASM 的优先级表可能和其他语言不一样。特别注意移位比加减法绑定更紧密：

```
// 第一项先执行移位；第二项用括号明确要求先做加法。
const shift_first = 1 + 2 << 3
const grouped_shift = (1 + 2) << 3

dd(shift_first, grouped_shift);
```

第一个值是 **17**，因为计算方式是 $1 + (2 \ll 3)$ 。第二个值是 **24**。

在混合移位、算术、掩码或比较的表达式中用括号。括号能说明意图，也让后续修改更安全。

表达式错误

表达式无法产生明确的编译期值时，XIRASM 会报错。常见原因：

- 名字未定义
- 操作数的值类型不合适
- 除数为零
- 整数溢出或下溢
- 字段不存在
- 函数参数无效

这些错误会终止汇编。不会用零代替或忽略，否则可能静默破坏指令操作数或二进制布局。

下一章讲 **if**、**while** 和 **for** 如何根据表达式选择源码、重复执行源码。

[返回语言指南](#)

第 4 章：控制流

控制流在编译期执行

XIRASM 的控制流是编译期行为，用来决定哪些源码会执行、哪些源码会重复执行。它不会自动生成运行时分支或循环。

```
// 这个编译期选项决定输出调试标记还是发布标记。
const debug_build = false

if debug_build {
    db("DEBUG");
} else {
    db("RELEASE");
}
```

只有一个分支产生字节。**debug_build** 为 **false** 时，输出包含 **RELEASE**，生成的文件里没有运行时条件判断。

编译期控制流用来：

- 按目标选择源码
- 重复生成数据或指令
- 遍历编译期集合
- 计算表格和偏移量
- 包含可选的文件格式记录
- 验证源码配置

运行时控制流还是用普通的 ISA 指令，比如 x86 的 **jmp**、**call** 和条件分支。

if 和 else

if 条件必须产生布尔值：

```
// 根据数据大小选择一个单字节标记。
const payload_size = 32

if payload_size <= 64 {
    db(0x01);
} else {
    db(0xff);
}
```

选中的块在自己的作用域中执行。另一个块不产生输出，也不执行编译期操作。

else 块可以省略：

```
// 只有选项启用时才写出标记文本。
const include_marker = true

if include_marker {
    db("MARK");
}
```

需要多于两个分支时用 **else if**：

```
// 通过链式条件把数据大小划分为三个区间。
const payload_size = 32

if payload_size < 16 {
    db(1);
} else if payload_size < 64 {
    db(2);
} else {
    db(3);
}
```

输出 02。

选择 ISA 指令

if 块里可以包含 ISA 指令：

```
// 选择 64 位 x86 指令编码。
x86.use64();

// 编译期常量决定采用哪组返回值指令。
const return_zero = true

entry:
  if return_zero {
    // 清零 eax, 令例程返回零。
    xor eax, eax
  } else {
    // 只有条件为假时才把返回值设为一。
    mov eax, 1
  }
  ret
```

因为 return_zero 为 true，生成的指令是：

```
// 编译期条件只保留清零返回值的路径。
xor eax, eax
// 返回调用者。
ret
```

没选中的 mov 指令不会进入输出。这是编译期指令选择，不是运行时条件分支。

目标条件

if 条件里可以直接用目标查询：

```
// x86-64 目标使用 64 位指令编码模式。
if target.isa == .x86_64 {
  x86.use64();
}

// 根据当前位宽写出对应的文本标记。
if target.bits == 64 {
  db("wide");
} else {
  db("narrow");
}
```

target.isa 标识选择的 ISA 系列，target.bits 报告当前位宽。x86.use32()、x86.use64()、riscv.use32()、riscv.use64() 等模式调用会更新后续目标条件看到的位宽。

目标查询是专用条件值。直接在 if 条件中使用，不要先复制到普通绑定里。

遍历范围

已知迭代次数时用 for 配合 range(start, end)：

```
// 依次写出从起始值零到结束值四之前的每个索引。
for index in range(0, 4) {
  db(index);
}
```

输出：

```
00 01 02 03
```

起始值包含，结束值不包含。范围必须递增或为空，递减范围如 range(4, 0) 会报错。

循环绑定属于循环体：

```
// 每次迭代都在索引上加 0x10，再写出编码后的值。
```

```
for index in range(0, 3) {
  const encoded = index + 0x10
  db(encoded);
}
```

输出 **10 11 12**。**index** 和 **encoded** 都是每次循环的局部变量。

遍历列表

for 循环可以遍历编译期列表的值：

```
// 列表按输出顺序保存三个待写出的字节值。
const opcodes: list = list.of(0x90, 0x90, 0xc3)

for opcode in opcodes {
  db(opcode);
}
```

按列表顺序遍历。适用于字节表、导入描述、生成的名字等集合数据。

映射不能直接遍历。用 **map.keys(...)** 或 **map.values(...)** 取得列表后再遍历。集合在第 6 章介绍。

while

当循环终止依赖循环内更新的值时用 **while**：

```
// 每次写出当前值，然后推进到下一个值。
let value = 1

while value <= 4 {
  db(value);
  value = value + 1
}
```

输出：

```
01 02 03 04
```

每次迭代前判断条件。初始为 **false** 则不执行循环体。

应让终止条件靠近循环更新逻辑，并且清晰可见。编译期循环如果停不下来会阻止汇编完成，所以 XIRASM 限制最多 1,000,000 次迭代，超限报错。

break 和 continue

break 结束当前最内层的 Meta 循环。**continue** 跳过本次迭代的剩余部分，开始下一次。两者都能用在 **for**、**while** 以及延迟块的 **while** 中，也包括嵌套的 **if** 语句。

例如：

```
for value in range(0, 8) {
  if (value == 6) {
    break;
  }
  if (value & 1) != 0 {
    continue;
  }
  db(value);
}
```

输出 **00 02 04**。循环控制语句不能跨函数调用边界，在循环外使用会报错。

选择控制流形式

需求	使用
两个源码块二选一	if / else

需求	使用
多个源码块选一个	<code>if / else if / else</code>
按 ISA 或位宽选源码	目标条件
重复固定次数	<code>for + range</code>
遍历编译期集合	<code>for + 列表</code>
重复到条件满足为止	<code>while</code>
选择生成的指令	<code>if</code> 包含 ISA 指令
提前结束最内层循环	<code>break</code>
跳过当前迭代的剩余部分	<code>continue</code>

迭代次数已知时优先用 `for`，比 `while` 更容易看出输出规律，也不需要手动维护循环变量。

下一章把重复的计算和输出操作打包成函数，带参数、返回值和局部作用域。

返回语言指南

第 5 章：函数与作用域

函数封装编译期工作

函数封装可复用的编译期逻辑。它可以计算值、写出数据，或组合多个底层操作。

XIRASM 有两种函数：

- 过程函数执行操作，不产生表达式值。
- 返回值函数计算值，用 `->` 声明返回类型。

两种函数都使用 `fn`、参数列表和代码块。函数在汇编期间执行；声明函数本身不产生字节，只有调用过程函数，或把返回值交给输出接口时，才会影响输出。

让一个函数只承担一种职责：要么执行输出或布局操作，要么返回一个可用于表达式的值。

过程函数

过程函数组合输出操作：

```
// 把传入字节及其后继值作为一组连续数据写出。
fn emit_pair(value: u8) {
    db(value);
    db(value + 1);
}

// 两次调用分别生成 02 03 和 08 09。
emit_pair(2);
emit_pair(8);
```

输出：

02 03 08 09

没有 `->` 返回类型的函数就是过程函数。过程调用是一条语句，末尾要加分号。

过程函数体可以包含声明、赋值、控制流、数据输出以及其他过程调用。它适合封装重复的二进制记录、指令序列、表项和格式构建步骤。

过程函数本身不是值，不能用于初始化绑定：

```
fn emit_marker() {
    db(0x90);
}

const marker = emit_marker()
```

这会报错，因为 `emit_marker()` 只执行操作，不返回表达式值。

参数和实参

参数写在括号里：

```
// 连续写出 count 个相同字节。
fn emit_run(value: u8, count: u64) {
    // index 负责控制循环次数，循环体只需要使用 value。
    for index in range(0, count) {
        db(value);
    }
}

// 写出四个 cc 字节。
emit_run(0xcc, 4);
```

输出四个 `cc` 字节。`index` 控制循环次数，即使循环体里未使用它。

参数类型可以省略：

```
// 形参类型可以省略，调用时传入的值会绑定到对应形参。
fn add(left, right) -> u64 {
    return left + right;
}

// 计算 20 + 22，并把结果 2a 写成一个字节。
db(add(20, 22));
```

输出 **2a**。实参值绑定到对应参数。公开的辅助函数建议加类型注解，让约定更明确，也能在调用处发现不合适的实参。

实参按位置对应参数。调用时必须为每个参数提供一个实参；没有默认实参机制。同一函数的参数名不能重复。每次调用有独立的参数绑定，互不影响。

返回值函数

函数需要产生表达式值时加 **-> type**：

```
// 把 value 向上对齐到 alignment 的整数倍。
fn align_up(value: u64, alignment: u64) -> u64 {
    return (value + alignment - 1) / alignment * alignment;
}

// 0x73 按 0x20 对齐后得到 0x80，再以双字节写出。
const header_size = align_up(0x73, 0x20)
dw(header_size);
```

函数返回 **0x80**，输出：

```
80 00
```

return 末尾加分号。表达式转换为声明的返回类型。返回类型可以是整数、布尔值、字符串、字节序列等：

```
// 判断传入值是否等于一个 0x1000 字节的页大小。
fn is_page(value: u64) -> bool {
    return value == 0x1000;
}

// 返回固定的两个签名字节。
fn signature() -> bytes {
    return b"XR";
}

// 先检查计算结果，再写出签名字节。
assert(is_page(0x1000));
db(signature());
```

返回值函数可以用在任何能接受其返回类型的位置：声明、实参、条件、函数调用或更大的表达式。

返回规则和副作用

返回值函数必须执行到 **return**。末尾无返回值会报错：

```
fn incomplete(value: u64) -> u64 {
    const doubled = value * 2
}

const result = incomplete(4)
```

返回值类型必须匹配：

```
fn enabled() -> bool {
    return 1;
}

const result = enabled()
```

报错，整数不满足 **bool** 约定。

返回值函数用于计算，不能输出数据或产生其他副作用：

```
fn bad_counter() -> u64 {
    db(1);
    return 1;
}

const result = bad_counter()
```

需要改变输出或布局时用过程函数，需要计算值时用返回值函数。

过程函数不声明返回类型，也不能返回值：

```
fn emit_one() {
    return 1;
}

emit_one();
```

过程函数执行到末尾即结束。

函数局部作用域

参数和函数内的绑定只属于当前调用：

```
// 加上固定开销，并确保结果不小于 16。
fn adjusted_size(size: u64) -> u64 {
    const overhead = 4
    let result = size + overhead

    if result < 16 {
        result = 16
    }

    return result;
}

// 两次调用各自使用独立的局部绑定。
db(adjusted_size(3));
db(adjusted_size(20));
```

输出 **10 18**。每次调用创建新的 **size**、**overhead**、**result**，调用结束后这些名字不再可用。

函数内的块创建嵌套作用域，可遮蔽外部名字：

```
// 使用嵌套作用域中的同名常量参与一次局部计算。
fn combine(value: u64) -> u64 {
    let result = value

    {
        // 此处的 value 只在这个代码块内表示常量 5。
        const value = 5
        result = result + value
    }

    return result;
}

// 参数值 3 与局部常量 5 相加，结果为 08。
db(combine(3));
```

输出 **08**。嵌套块中 **value** 指向局部常量 **5**。块结束后参数 **value** 重新可见。

中间计算用局部名字，避免临时状态泄漏到外部，多次调用互不干扰。

声明顺序

函数必须在调用前声明：

```
// 先声明函数，后面的源代码才能引用它。
fn add(left: u64, right: u64) -> u64 {
    return left + right;
}
```

```
// 调用已经可见的函数，并写出结果。  
const answer = add(20, 22)  
db(answer);
```

把调用移至声明前会报错，此时函数名尚未定义。

函数声明只能出现在顶层，不能写在另一个函数、循环、条件块或普通块内。相关函数可以在顶层相邻排列，供后续源码调用。

第 10 章介绍如何通过包含文件将函数声明提供给另一个源文件。

递归和调用深度

返回值函数在有终止条件时可以递归调用自身：

```
// 递归计算从 value 到 1 的总和。  
fn triangular(value: u64) -> u64 {  
    // value 为零时停止递归。  
    if value == 0 {  
        return 0;  
    }  
  
    return value + triangular(value - 1);  
}  
  
// triangular(4) 计算 4 + 3 + 2 + 1，并写出 0a。  
db(triangular(4));
```

输出 0a，即 4 + 3 + 2 + 1 的结果。

递归在编译期执行。递归深度应控制在一定范围内，终止条件必须明确。XIRASM 拒绝超过 128 层调用的函数链，防止失控递归。

遍历范围或集合时用循环。只有计算本身具有递归结构且终止条件明确时，才用递归。

选哪种函数

需求	使用
输出字节或指令	过程函数
组合多个输出调用	过程函数
为表达式计算值	返回值函数
复用纯计算逻辑	返回值函数
临时名字不泄漏	两种均可
遍历范围或集合	通常用循环
递归计算	递归返回值函数

每个函数专注一件事。小计算函数易于组合，小过程函数使输出效果在调用处可见。

下一章介绍列表、映射、字符串和字节序列，供需要处理集合和文本的函数使用。

返回语言指南

第 6 章：集合与文本

集合是编译期值

XIRASM 有四种编译期值类型：

类型	表示什么	常见用途
<code>string</code>	编译期文本	名字、路径、错误信息、文本字段
<code>bytes</code>	确切的二进制数据	签名、魔数、已编码指令字节、打包记录
<code>list</code>	有序的值序列	重复表项、参数列表、配置条目
<code>map</code>	字符串到值的映射	配置选项、记录字段、查找表

这些类型本身不会写出数据。只有把它们传给输出接口，数据才会进入输出文件。

字符串、字节序列、列表和映射都是编译期值。普通 API 会产生新值；当局部算法需要逐步构建 `list` 或 `map` 时，应把集合放在 `let` 绑定中，并使用 `list.push_mut`、`list.set_mut`、`map.set_mut` 这类语句 API 直接更新它。值在传递时会复制，调用者之间不会共享可变内存。

字符串表示源文本

字符串存放编译期使用的文本：

```
// 先清除名称两端的空白，再把 ASCII 字母统一转换为小写。
const raw_name: string = " Kernel64 "
const name: string = lower(trim(raw_name))

// 检查规范化后的名称及其组成部分。
assert(name == "kernel64");
assert(starts_with(name, "kernel"));
assert(ends_with(name, "64"));
assert(contains(name, "nel"));

// 把最终文本作为字节写入输出内容。
emit.bytes(name);
```

输出 `kernel64` 的八个文本字节。

常用字符串 API：

- `trim(text)` 去掉两端空白
- `lower(text)`、`upper(text)` 转 ASCII 字母大小写
- `starts_with`、`ends_with`、`contains` 检测文本
- `replace(text, needle, replacement)` 替换匹配文本
- `to_string(value)` 把值转成文本

大小写转换只处理 ASCII。名字、路径、错误消息、配置键适合字符串。二进制数据用 `bytes`。

分割和连接文本

`split` 按分隔符拆成字符串列表，`join` 把列表连成文本：

```
// 把逗号分隔的节名称拆成列表，再按路径形式重新连接。
const sections: list = split("text,data,bss", ",")
const path: string = join(sections, "/")

// 列表索引从零开始，因此索引 0 和 2 分别对应首尾元素。
assert(len(sections) == 3);
assert(list.get(sections, 0) == "text");
assert(list.get(sections, 2) == "bss");
assert(path == "text/data/bss");
```

```
// 写出重新组合后的路径文本。  
emit.bytes(path);
```

索引从零开始。这种写法适合处理小配置列表、格式选项中的文本，以及动态生成的名字。

用字节序列表示二进制值

bytes 是一段确切的字节序列：

```
// 从十六进制文本构造文件标记，并把版本号编码为两个小端字节。  
const magic: bytes = bytes.from_hex("58495200")  
const version: bytes = bytes.le(3, 2)  
const header: bytes = bytes.concat(magic, version)  
  
// 检查原始标记和版本字段的编码结果。  
assert(bytes.eq(magic, bytes.from_hex("58495200")));  
assert(bytes.hex(version) == "0300");  
  
// 按连接后的顺序写出完整文件头。  
emit.bytes(header);
```

输出 **58 49 52 00 03 00**。

bytes.from_hex 把十六进制文本转二进制。**bytes.le(value, width)** 用指定字节数按小端序编码整数。

bytes.concat 拼接字节序列。**bytes.eq** 比较两个序列。**bytes.hex** 把二进制显示为小写十六进制文本。

构建字节序列

字节序列操作会返回新值，原值不变：

```
// 从 ABC 的字节表示开始，在索引 1 处插入连字符。  
const base: bytes = bytes.from_hex("414243")  
const marked: bytes = bytes.insert(base, 1, b"-")  
// 从索引 2 开始替换一个字节，并追加两个 0xff 字节。  
const patched: bytes = bytes.replace(marked, 2, 1, b"Z")  
const trailer: bytes = bytes.repeat(2, 0xff)  
const result: bytes = bytes.concat(patched, trailer)  
  
// 只写出最终结果，各个中间值仍可继续复用。  
emit.bytes(result);
```

输出 **41 2d 5a 43 ff ff**。

操作包括：

- **bytes.new()** 空序列
- **bytes.push(value, byte)** 末尾追加
- **bytes.repeat(count, byte)** 重复字节
- **bytes.insert(value, index, addition)** 在零开始索引处插入
- **bytes.replace(value, index, count, replacement)** 替换范围
- **bytes.concat(left, right)** 拼接

原始 **base** 仍然是 **41 42 43**。每次操作产生新值，旧值不受影响。

列表保持顺序

列表存放有序值：

```
// 先追加一个元素，再替换索引 1 处的值。  
let items: list = list.of(1, 2, 3)  
list.push_mut(items, 4);  
list.set_mut(items, 1, 0xaa);  
// 从索引 1 开始截取两个元素，原列表不会被修改。  
const middle: list = list.slice(items, 1, 2)
```

```
assert(list.eq(items, list.of(1, 0xaa, 3, 4)));
assert(list.eq(middle, list.of(0xaa, 3)));

for value in items { db(value); }
```

输出 **01 aa 03 04**。

`list.push_mut` 和 `list.set_mut` 直接更新 `items` 这个 `let` 绑定。`list.slice` 返回从起始索引开始的指定个元素，不修改原列表。

其他：`list.new()`、`list.get()`、`list.concat()`、`list.eq()`、`len()`。

更新 `let` 绑定的集合

一个局部算法需要逐步构建列表或映射时，使用可变语句 API：

```
let items: list = list.of(1, 2)
list.push_mut(items, 3);
list.set_mut(items, 0, 4);

let options: map = map.new()
map.set_mut(options, "arch", "x64");
map.set_mut(options, "items", items);
```

第一个参数必须是直接的 `let` 绑定标识符。目标不能是 `const`、临时表达式、函数返回值、字段访问、缺失名称，或类型不匹配的集合。词法查找会使用最近的绑定，因此块内 `let` 会遮蔽外层同名绑定。顶层 `let` 可以在普通源码处理阶段更新；返回值函数只能更新本次调用内部的局部 `let`。

`list.push_mut` 追加一个复制后的元素。`list.set_mut` 替换已有的零基索引。`map.set_mut` 插入或替换字符串键对应的值；替换已有键时保留原有插入位置。这些语句不会产生表达式值，也不能出现在 `defer` 或 `late_layout` 块中。

值复制有明确边界：

```
let items: list = list.of(1)
const snapshot: list = items
list.push_mut(items, 2);

assert(list.eq(snapshot, list.of(1)));
assert(list.eq(items, list.of(1, 2)));
```

列表可以保存更大的值

列表元素不限于整数，也可以是字符串、字节序列、映射等更大的编译期值：

```
// 把文本标记和一个双字节小端整数组织成有序字节块。
const chunks: list = list.concat(
  list.of(b"XR"),
  list.of(bytes.le(0x1234, 2))
)

for chunk in chunks { emit.bytes(chunk); }
```

输出 **58 52 34 12**。

映射保存命名值

映射把字符串键关联到值。逐步构建映射时，直接使用 `map.set_mut` 更新 `let` 绑定：

```
// 逐步加入命名属性，并用新值替换已有的 arch 属性。
let complete: map = map.new()
map.set_mut(complete, "arch", "x64");
map.set_mut(complete, "mode", "release");
map.set_mut(complete, "arch", "rv64");
// 映射中的值也可以是列表等更大的编译期值。
map.set_mut(complete, "tags", list.of("asm", "dsl"));

// 检查键是否存在，并读取必需或带默认值的属性。
assert(len(complete) == 3);
assert(map.has(complete, "arch"));
```

```
assert(!map.has(complete, "missing"));
assert(map.get(complete, "arch") == "rv64");
assert(map.get_or(complete, "missing", "default") == "default");
assert(list.eq(map.get(complete, "tags"), list.of("asm", "dsl")));
```

映射键必须是字符串。添加新键会追加一个条目；替换已有键会更新值，但保留这个键原来的位置。`map.keys` 和 `map.values` 因此会返回互相对应的并行列表。

遍历映射内容

映射用于按键查找。需要遍历时先用 `map.keys` 或 `map.values` 转列表：

```
// 用映射保存两个带名称的二进制字段。
let fields: map = map.new()
map.set_mut(fields, "magic", b"XR");
map.set_mut(fields, "version", bytes.le(3, 2));
// 键和值分别转换为列表，便于检查数量或逐项处理。
const keys: list = map.keys(fields)
const values: list = map.values(fields)
assert(len(keys) == 2);
assert(len(values) == 2);
// 逐项写出映射中保存的字节序列。
for value in values {
    emit.bytes(value);
}
```

文件格式规定了确切顺序时，用列表保存顺序。映射主要负责按名称查找。

在函数中组合集合

集合转换可以封装为函数：

```
// 把列表中的每个数值编码为两个小端字节。
fn encode_u16(values: list) -> bytes {
    let result: bytes = bytes.new()
    // 每次连接都会产生新的字节序列，并更新可变绑定 result。
    for value in values {
        result = bytes.concat(result, bytes.le(value, 2))
    }
    return result;
}

// 调用者决定何时把函数生成的字节序列写入输出内容。
const words: list = list.of(0x1234, 0xabcd)
emit.bytes(encode_u16(words));
```

输出 `34 12 cd ab`。函数逐步构建字节序列，调用者决定何时把结果写入输出。

职责划分：字符串保存名字和文本，映射保存命名配置，列表保存有序值，字节序列保存最终二进制表示，过程函数负责决定何时写出。

选哪种集合

需求	使用
面向人的源文本	<code>string</code>
确切的二进制表示	<code>bytes</code>
有序值序列	<code>list</code>
按字符串键查找	<code>map</code>
处理按分隔符拆分的文本	<code>split</code> 转 <code>list</code>
重新组合分隔文本	<code>join</code>
逐字节构建二进制记录	<code>bytes</code> API
既要顺序又要查找	<code>list</code> + <code>map</code>

下一章讲词法元素和模式匹配，用于需要按语法解析的源文本。

[返回语言指南](#)

第 7 章：词法单元与模式匹配

本章讨论在编译期检查“类似源码的文本”。字符串 API 适合处理普通文本；当你关心名称、标点、运算符、字面量和括号分组这些源代码结构时，应先把文本拆成词法单元，再进行模式匹配。

不要为了汇编普通 ISA 指令而手动分词。自然 ISA 文本已经是 XIRASM 源语言的一部分，可以直接写在源码里。词法单元匹配主要用于小型用户 DSL、生成的源码片段、小型命令记录和可复用的编译期辅助函数。

词法单元表示源代码结构

字符串辅助接口把文本视为字符序列。对于名称、路径、分隔字段和简单替换，这种模型已经够用。类似源码的文本通常需要另一种模型。

请看下面这段文本：

```
load rax, [rbx+(rcx*4)]
```

其中的逗号、方括号、圆括号、名称、运算符和整数字面量都具有结构意义。如果只按空格拆分，不仅会丢失这些结构，还会对仅有空格差异的等价写法产生不一致的处理结果。

XIRASM 可以把类似源代码的文本转换为词法单元列表：

```
// 把表达式拆成词法单元，并验证空白不会进入结果列表。
const source: string = "count + 0x2a"
const source_tokens: list = tokens.of(source)

assert(len(source_tokens) == 3);
assert(list.get(source_tokens, 0) == "count");
assert(list.get(source_tokens, 1) == "+");
assert(list.get(source_tokens, 2) == "0x2a");

// 按规范形式重新连接词法单元，并把结果写入输出内容。
emit.bytes(tokens.join(source_tokens));
```

这段代码会写出规范化后的文本：

```
count+0x2a
```

`tokens.of` 会丢弃无意义的空白，同时保留词法单元的顺序。`tokens.join` 使用规范间距重新呈现这个序列，并不会逐字节还原原始字符串。

当字符排列本身很重要时，应使用字符串。当名称、标点、运算符、字面量和平衡分组的结构很重要时，应使用词法单元。

匹配词法单元结构

`match.tokens(pattern, input)` 会把类似源代码的输入与词法单元模式比较：

```
// 匹配 load 形式，并分别捕获目标名称和完整的源操作数。
const line: string = "load rax, [rbx+(rcx*4)]"
const result: map = match.tokens(
  "=load destination:name =, source:tokens",
  line
)

// 完整匹配成功后，再读取命名捕获得到的值。
assert(map.get(result, "ok"));

const captures: map = map.get(result, "captures")
assert(map.get(captures, "destination") == "rax");
assert(tokens.join(map.get(captures, "source")) == "[rbx+(rcx*4)]");
```

结果是一个包含两个字段的映射：

- `"ok"` 是布尔值，表示完整输入是否匹配成功；

- **"captures"** 是一个映射，保存各个命名捕获所提取的值。

只有检查 **"ok"** 后，才能读取捕获结果。

字面量模式词法单元

以 **=** 开头的模式片段会匹配一个完全相同的词法单元：

```
// `=&&` 要求输入中必须出现精确的逻辑与词法单元。
const result: map = match.tokens(
  "left:name =&& right:name",
  "ready && enabled"
)

assert(map.get(result, "ok"));
```

=&& 要求输入中出现逻辑与词法单元。同一规则也适用于名称和标点：

```
=load    精确匹配名称词法单元
=,       精确匹配逗号词法单元
=[       精确匹配左方括号词法单元
===      精确匹配相等运算符词法单元
```

第一个 **=** 是模式标记；**===** 表示“匹配 **==** 词法单元”，而不是匹配一个由三个字符组成的相等运算符。

模式片段之间使用空白分隔。模式中的空白只用于提高可读性；实际匹配依据的是词法单元，而不是输入中原有的间距。

捕获种类

捕获的形式为 **name:kind**。捕获名称会成为结果中 **"captures"** 映射的键。

种类	匹配内容	捕获的值
token	任意种类的一个词法单元	string
name	一个类似标识符的名称	string
int	一个整数字面量	整数值
quoted	一个带引号的词法单元	去除引号后的 string
tokens	一段保持分组平衡的词法单元	由词法单元字符串组成的 list

同一个模式中的捕获名称必须唯一。

单词法单元捕获可以精确描述小型命令语法：

```
// 从 set 形式中提取目标名称，并把整数字面量转换为整数值。
const assignment: map = match.tokens(
  "=set target:name =, value:int",
  "set count, 0x2a"
)
const assignment_captures: map = map.get(assignment, "captures")

// 保留中间运算符的原始词法单元文本。
const operation: map = match.tokens(
  "left:name operator:token right:name",
  "count + step"
)
const operation_captures: map = map.get(operation, "captures")

// 去除外层引号后捕获消息文本。
const message: map = match.tokens("db text:quoted", "db 'READY'")
const message_captures: map = map.get(message, "captures")

// 验证每种捕获都生成了预期类型和值。
assert(map.get(assignment, "ok"));
assert(map.get(assignment_captures, "target") == "count");
assert(map.get(assignment_captures, "value") == 42);
assert(map.get(operation_captures, "operator") == "+");
assert(map.get(message_captures, "text") == "READY");
```

int 捕获会把字面量转换为整数值。**quoted** 捕获会移除外层引号，并解码受支持的转义序列。**token** 捕获只返回词法单元文本，不会要求它属于更具体的类型。

平衡的词法单元范围

tokens 捕获可以使用多个词法单元，同时保持圆括号、方括号和花括号的平衡：

```
// 把方括号内的嵌套地址表达式作为一个平衡范围捕获。
const result: map = match.tokens(
  "=load destination:name =, address:tokens",
  "load rax, [rbx+(rcx*4)]"
)
const captures: map = map.get(result, "captures")
const address: list = map.get(captures, "address")

assert(map.get(result, "ok"));
assert(tokens.join(address) == "[rbx+(rcx*4)]");
```

捕获的值是词法单元列表，而不是字符串。继续保留词法单元形式，其他匹配器就能直接检查这段内容，无须再次对文本词法分析。

平衡捕获适用于 **()**、**[]** 和 **{}**。**<**、**>** 等比较运算符仍然只是普通的运算符词法单元：

```
// 比较运算符不会被当作分组边界，整个表达式仍可被捕获。
const result: map = match.tokens("expression:tokens", "left < right")
const captures: map = map.get(result, "captures")

assert(map.get(result, "ok"));
assert(tokens.join(map.get(captures, "expression")) == "left<right");
```

最小匹配与回溯

当 **tokens** 捕获后面还有其他模式片段时，它最初会尽可能少地使用词法单元。如果剩余模式无法匹配，捕获范围就会扩展，然后重新尝试匹配：

```
// prefix 会先尝试空范围，再扩展到足以让整数捕获成功的位置。
const result: map = match.tokens(
  "prefix:tokens value:int",
  "name 42"
)
const captures: map = map.get(result, "captures")

assert(map.get(result, "ok"));
assert(tokens.join(map.get(captures, "prefix")) == "name");
assert(map.get(captures, "value") == 42);
```

第一次尝试会给 **prefix** 分配一个空范围，但 **value:int** 无法匹配 **name**。匹配器随后扩展 **prefix**，使其包含 **name**，这样整数捕获就能匹配 **42**。

带类型的捕获遇到错误种类的词法单元时，只是一次普通的匹配失败。一个有效模式不会产生汇编错误：

```
// name 不是整数字面量，模式有效，但本次输入不匹配。
const result: map = match.tokens("value:int", "name")
assert(!map.get(result, "ok"));
```

这种行为便于依次尝试多个有效模式。

空词法单元范围

tokens 捕获可以为空：

```
// call 后没有参数，arguments 捕获得到一个空列表。
const result: map = match.tokens("=call arguments:tokens", "call")
const captures: map = map.get(result, "captures")

assert(map.get(result, "ok"));
assert(len(map.get(captures, "arguments")) == 0);
```

如果词法单元范围必须非空，可以在匹配成功后检查其长度，也可以在模式中加入另一个必需的捕获。

匹配已有词法单元列表

传给 `match.tokens` 的输入也可以是已经生成的词法单元字符串列表：

```
// 先一次词法分析，再把同一列表直接交给模式匹配器。
const input: list = tokens.of("load r1, 42")
const result: map = match.tokens(
  "=load destination:name =, value:int",
  input
)
const captures: map = map.get(result, "captures")

assert(map.get(result, "ok"));
assert(map.get(captures, "destination") == "r1");
assert(map.get(captures, "value") == 42);
```

当多个处理步骤需要检查同一段源代码时，可以在辅助接口之间直接传递词法单元列表。只有在确实需要文本值时，才使用 `tokens.join` 转回文本。

尝试不同结构

模式本身不包含“或者”运算符。应在编译期控制流中依次尝试每一种完整结构：

```
// 先尝试 load 结构；当前输入不匹配时，再尝试 store 结构。
const line: string = "store r1, r2"
const load: map = match.tokens(
  "=load destination:name =, source:name",
  line
)

if map.get(load, "ok") {
  // load 分支只写出目标名称。
  const captures: map = map.get(load, "captures")
  emit.bytes(map.get(captures, "destination"));
} else {
  const store: map = match.tokens(
    "=store destination:name =, source:name",
    line
  )

  // store 分支写出目标名称和源名称。
  assert(map.get(store, "ok"));
  const captures: map = map.get(store, "captures")
  emit.bytes(map.get(captures, "destination"));
  emit.bytes(map.get(captures, "source"));
}
```

这段代码会写出 `r1r2`。每个模式只描述一种有效形式，普通的 `if/else` 逻辑负责判断输入采用了哪一种形式。

对于更大的语法，可以把每种结构放进一个返回值小函数中，并让调用位置清楚地显示分派顺序。

匹配失败与无效模式

普通的结构或类型不匹配会返回 `"ok" == false` 的结果。比如：

- 精确字面量不同；
- `name` 捕获收到标点；
- `int` 捕获收到名称；
- `tokens` 捕获中的括号不平衡；
- 模式已经结束，但输入仍有剩余。

格式错误的模式则不同。它会作为汇编错误被拒绝，而不是被报告为匹配失败。未知的捕获种类、空字面量或重复的捕获名称都会造成模式错误：

```
const invalid = match.tokens(
  "value:name value:int",
```

```
"left 42"  
)
```

两个捕获都使用了键 `"value"`，这个模式无效。

请记住两者的区别：

- 模式有效，但输入不适用，表示 `"ok" == false`；
- 模式定义无效，表示源代码有错误，应当修正模式。

字符串还是词法单元

需求	使用方式
搜索子字符串	<code>contains</code>
检查文本前缀或后缀	<code>starts_with</code> / <code>ends_with</code>
解析使用简单分隔符分开的文本	<code>split</code>
保留名称、标点和分组结构	<code>tokens.of</code>
匹配精确的源代码结构	<code>match.tokens</code>
提取带类型的字段	<code>name</code> 、 <code>int</code> 或 <code>quoted</code> 捕获
捕获嵌套表达式或操作数	<code>tokens</code> 捕获
尝试多种命令形式	配合 <code>if</code> / <code>else</code> 使用多个模式

不要仅仅为了汇编普通 ISA 指令，就先把指令行转换为词法单元。自然 ISA 文本本来就是源语言的一部分，可以直接书写。词法单元匹配最适合用于紧凑的用户自定义 DSL、生成的源代码片段、小型命令记录，以及可复用的编译期辅助函数。

至此，本指南的语言基础部分全部结束。下一章将从目标、自然 ISA 文本、标号和引用开始介绍汇编器模型。

[返回语言指南](#)

第 8 章：目标、ISA 文本与标号

编译期语言决定生成什么；汇编器模型决定生成的指令和数据放在哪里、由哪个指令集编码，以及符号引用如何变成具体值。

本章从三个紧密相关的概念开始：

- 当前目标选择 ISA 和指令宽度；
- 自然 ISA 文本为当前目标创建指令片段；
- 标号为这些片段提供稳定的符号名称。

手写汇编时，这些功能应该很直接：选择目标、写指令、正常使用标号。只有确实需要编译期生成时，才使用动态指令文本和动态标号名。

当前目标

每条指令都会按当前目标编码。目标包含 ISA 系列，以及该 ISA 编码所需的位宽等信息。

命令行选项设置初始目标：

```
xirasm program.xir -o program.bin --target x86-64
xirasm program.xir -o program.bin --target x86
xirasm program.xir -o program.bin --target rv64
xirasm program.xir -o program.bin --target rv32
xirasm module.spvasm -o module.spv --target spv
```

没有显式选择时，XIRASM 默认使用 64 位 x86。源码仍然可以写明指令模式；示例、可复用 include 和依赖特定位宽的代码都建议显式声明：

```
// 明确选择后续指令使用的 64 位 x86 编码。
x86.use64();

entry:
// 生成一个返回零的最小指令序列。
xor eax, eax
ret
```

模式调用只影响它之后出现的指令。每条指令都会记住创建它时处于活动状态的目标。

选择 x86、RISC-V 和 SPIR-V

源文件内用这些接口切换模式：

接口	后续指令模式
<code>x86.use16()</code>	16 位 x86
<code>x86.use32()</code>	32 位 x86
<code>x86.use64()</code>	64 位 x86
<code>riscv.use32()</code>	32 位 RISC-V
<code>riscv.use64()</code>	64 位 RISC-V
<code>spv.use()</code>	SPIR-V 1.6 模块

同一源文件可以切换模式：

```
// 第一条指令使用 16 位 x86 编码。
x86.use16();
mov ax, 1

// 第二条指令使用 32 位 x86 编码。
x86.use32();
mov eax, 2

// 第三条指令使用 64 位 x86 编码。
```

```
x86.use64();
mov rax, 3
```

这三条指令分别保留 16 位、32 位和 64 位编码上下文。模式调用不会改写之前已经创建的指令。

选择编码模式不等于让生成的程序在运行时切换处理器模式。引导映像、内核、固件组件或混合模式程序仍然需要自己安排运行时模式转换。RISC-V 宽度选择也遵循同样的源码顺序规则：

```
// 选择 XLEN 为 64 位的 RISC-V 模式。
riscv.use64();

// 两条指令都会按照当前 64 位 RISC-V 目标编码。
addi x1, x0, 1
addi x0, x0, 0
```

不要把 x86 的模式概念套到 RISC-V 或 SPIR-V 上。XIRASM 会分别保存各 ISA 的目标设置，而不是把所有 ISA 都压成一个通用的 `mode_bits` 值。

SPIR-V 不是逐条独立编码的指令流，而是一个完整的逻辑模块。用 `spv.use()` 选择 SPIR-V 1.6，然后直接书写标准 `Op*` 指令和数字结果 ID：

```
spv.use();

OpCapability Shader
OpMemoryModel Logical GLSL450
%1 = OpTypeVoid
```

命令行可用 `--target spv` 或 `--target spirv`，两者都选择 SPIR-V 1.6。同一个 SPIR-V 输出只能包含同一个 section、同一个模块版本的 SPIR-V ISA 行，不能混入 x86/RISC-V 指令，也不能混入数据写出、预留或对齐片段。结果 ID 目前必须写成 `%1` 这类数字形式，不接受符号 ID。

查询目标

编译期控制流可以检查当前目标：

```
// 选择 x86 后端和 64 位模式。
x86.use64();

// 这个条件在汇编期间判断，不会生成运行时分支。
if target.isa == .x86_64 {
    assert(target.bits == 64);
    mov eax, 1
}
```

可查询的目标系列值：

值	目标系列
<code>.x86_64</code>	x86 (use16/use32/use64)
<code>.riscv64</code>	RISC-V (xlen 32/64)
<code>.spirv</code>	SPIR-V

这些公开的目标系列名称标识后端系列，不报告当前指令宽度。例如 `x86.use32()` 之后，`target.isa == .x86_64` 仍然成立，但 `target.bits == 32`。

需要判定位宽时用 `target.bits`；RISC-V 条件也可以使用 `target.xlen`：

```
// 选择 XLEN 为 32 位的 RISC-V 模式。
riscv.use32();

// 同时检查后端系列与 XLEN。
if target.isa == .riscv64 {
    assert(target.xlen == 32);
    addi x1, x0, 1
}
```

自然 ISA 文本

指令使用所选 ISA 的常规文本语法：

```
// 选择 64 位 x86，然后直接编写普通指令。  
x86.use64();
```

```
entry:  
    mov rax, 1  
    add rax, 2  
    ret
```

ISA 行由助记符和操作数组成。空格用于提高可读性；方括号、圆括号或花括号中的逗号属于嵌套操作数的一部分，不会把指令错误拆开。

ISA 行末尾不写分号。结构化的编译期调用需要分号：

```
// 这两行是编译期接口调用，因此以分号结束。  
x86.use64();  
emit.u8(0x90);
```

```
// 这一行是 ISA 文本，因此不写分号。  
nop
```

`x86.use64();` 和 `emit.u8(0x90);` 是编译期 API 调用，`nop` 是自然 ISA 文本。

XIRASM 会把指令文本和当前目标一起保存，再交给对应 ISA 后端编码。标号、布局、输出区域和 fixup 仍由前端负责。

SPIR-V 是逐指令编码规则的例外：汇编器会按源码顺序收集整个模块的指令，再一次性交给后端，使模块头、ID 上界、类型上下文和扩展指令集保持一致。

在指令中使用编译期值

编译期常量可以直接出现在指令操作数中：

```
// 在汇编期间计算立即数。  
x86.use64();  
const initial_value: u32 = 40 + 2  
  
entry:  
    // 编译期常量会直接成为指令操作数。  
    mov eax, initial_value  
    ret
```

标号也可以在指令中参与运算：

```
// 把逻辑起始地址设置为 0x1000。  
x86.use64();  
origin(0x1000);  
  
target:  
    // 操作数表示标号地址再加四。  
    mov rax, target + 4  
    ret
```

汇编器会以符号形式保留表达式，等地址确定后再求值。

只有确实需要动态生成时，才拼接指令字符串。

静态标号

标号写在名字后加冒号：

```
// 使用静态标号表达普通控制流。  
x86.use64();  
  
entry:  
    mov eax, 1  
    jmp done  
  
done:  
    ret
```

标号不产生任何字节，只把名字绑定到当前位置。

标号可以先使用后定义：

```
// finished 在跳转指令之后定义，仍然可以提前引用。
x86.use64();

entry:
  jmp short finished
  mov eax, 1

finished:
  ret
```

前向引用的长度选择属于指令编码约束。源码可以显式写出 **short** 或 **near**，也可以让布局和后端在约束允许时处理符号位移；不要在源码中手算跳转偏移。

符号引用与地址回填

包含符号的指令宽度不固定。

XIRASM 的处理顺序：

- 源文件定义标号并生成指令片段。
- 后端逐条编码指令，符号字段先保持未解析。
- 布局器给标号和片段分配最终地址。
- 地址确定后计算每个标号表达式，回填已编码字段。

前向引用不需要手动计算地址：

```
// 跳转目标稍后定义，位移由汇编器计算。
x86.use64();

entry:
  jmp target
  nop

target:
  ret
```

只要源文件描述指令及其标号表达式，而非手动填偏移，前面的指令改变长度也不会失效。

动态指令文本与动态标号

编译期需要计算名字时使用动态指令和标号：

```
// 根据多个固定部分组成动态标号名称。
const done: string = sym.join("generated_", "done")

// 动态指令和动态标号仍然进入正常的汇编流程。
isa(sym.join("jmp ", done));
label.define(done);
isa("ret");

// 这是上一个动态示例所对应的普通静态写法。
jmp generated_done
generated_done:
ret
```

isa(text) 把指令文本交给当前 ISA 后端编码，**label.define(name)** 在当前位置创建标号。命名用 **sym.join**，展开用 **sym.unique**。

只在确实需要时用动态标号。不要替代 **loop:**、**done:** 等静态写法。

读取标号地址

label_addr(label_or_name) 返回标号的逻辑地址：

```
// 定义一个普通标号，并把它的逻辑地址写入输出。  
x86.use64();  
  
entry:  
    ret  
  
dq(label_addr(entry));
```

指令内的标号引用通过地址计算完成，不需要手动查询。**label_addr()** 用于数据段写入标号地址这类求值场景。

布局未稳定时改用 **defer**（第 12 章）延迟查询。需要重定位时用格式接口，不要手动填偏移。

实用规则

- 源文件开头明确选择目标模式。
- 指令用自然 ISA 文本。
- 编译期调用末尾加分号，指令末尾不加。
- 源文件中地址明确时用静态标号。
- 前向引用以符号形式传递地址。
- 只在确实需要时用 **isa** 和 **label.define**。
- 判定位宽用 **target.bits**，判定系列用 **target.isa**。
- 布局未稳定时，推迟绝对地址查询到 **defer**。
- 需要可加载文件中的重定位时，使用格式接口提供的重定位声明。

下一章介绍数据声明和二进制布局。

[返回目录](#)

第 9 章：数据与二进制布局

指令不是输出字节的唯一来源。文件头、查找表、消息字面量、预留空间，都需要明确的二进制布局。

XIRASM 提供两层：

- 直接写数据的接口
- 定义结构体/联合体的类型系统

几行数据直接用写接口就够了。常用记录格式定义成结构体。

输出内容是有序字节序列

数据从当前位置开始追加：

```
// 按 1、2、4、8 字节的宽度依次写出整数。
emit.u8(0x11);
emit.u16(0x2233);
emit.u32(0x44556677);
emit.u64(0x0102030405060708);
```

按小端序写：

```
11
33 22
77 66 55 44
08 07 06 05 04 03 02 01
```

值超出目标宽度会报错，不会静默截断；需要更大的字段时，应选择更宽的写出接口。

简写版本：

简写	宽度	等效用途
db	1 字节	字节值、字符串和 bytes 值
dw	2 字节	小端整数
dd	4 字节	小端整数
dp	6 字节	小端整数
dq	8 字节	小端整数
dt	10 字节	从 u64 零扩展的小端整数
ddq	16 字节	从 u64 零扩展的小端整数
dqq	32 字节	从 u64 零扩展的小端整数
ddqq	64 字节	从 u64 零扩展的小端整数

例如：

```
// 使用简写按固定宽度写出同一组小端整数。
db(0x41);
dw(0x1122);
dd(0x33445566);
dq(0x0102030405060708);
```

简写接受多个参数。**db** 还能接字符串和字节序列，其他简写只接整数。1、2、4、6、8 字节写出会检查范围；10、16、32、64 字节写出会从 **u64** 零扩展。需要精确位模式时用 **emit.bytes**。**dt** 只是 10 字节整数数据写出形式，不表示 x87 f80 浮点格式。

浮点值

```
const scale: f64 = 1.5
const compact: f32 = f32(scale)
emit.f32(compact);
```

```
emit.f64(scale * 2.0);
```

emit.f32/emit.f64 写出 IEEE-754 binary32/binary64。实参类型必须精确匹配：**emit.f32** 接受 **f32**，**emit.f64** 接受 **f64**。溢出、NaN 和 Infinity 都会报错；有限下溢和有符号零会按 IEEE-754 位模式保留。

字符串与字节序列

一次 **db** 调用可以同时组合字节值、字符串和 **bytes** 值：

```
// suffix 保存需要原样接在文本之后的两个字节。
const suffix: bytes = b"CD"

// 依次写出字节 A、字符串 B、字节序列 CD 和显式终止零。
db(0x41, "B", suffix, 0);
```

输出：

```
41 42 43 44 00
```

字符串和字节序列原样复制。XIRASM 不自动加 NUL。要终止符自己写。

emit.bytes(value) 写出字符串或 **bytes** 值：

```
// 从十六进制文本构造精确的四字节签名。
const signature: bytes = bytes.from_hex("7f454c46")

// 先写出二进制签名，再直接写出四个文本字节。
emit.bytes(signature);
emit.bytes("DATA");
```

当一个值表示精确的二进制内容时，应使用 **bytes**。当一个值表示源代码层面的文本，只是恰好需要直接写出时，应使用字符串。

预留空间

reserve(count) 会让输出位置前进指定的字节数：

```
// 在两个已初始化字节之间预留两个字节。
db(0xeb);
reserve(2);
db(0xfe);
```

在普通 flat binary 输出中，这会写出：

```
eb 00 00 fe
```

预留简写把数量乘以元素宽度：

简写	预留字节数
rb(count)	count
rw(count)	count * 2
rd(count)	count * 4
rp(count)	count * 6
rq(count)	count * 8
rt(count)	count * 10
rdq(count)	count * 16
rqq(count)	count * 32
rdqq(count)	count * 64

```
// 预留两个字节，写出 aa，再预留两个字节并写出 bb。
rb(2);
db(0xaa);
rw(1);
db(0xbb);
```

这会依次写出两个零字节、**aa**、另外两个零字节和 **bb**。

预留操作表达的是未使用的空间，而不是具有实际意义的已初始化内容。如果后面还有已初始化的输出内容，预留范围会在 flat binary 中成为真实的零填充间隙。如果连续的预留空间位于末尾，则可能只增加逻辑大小，而不占用文件字节。第 11 章会通过输出区域的实际文件光标和潜在文件光标解释这一区别。

填充与对齐

pad(count, fill) 写 count 个重复字节：

```
// 在 01 与 02 之间写出三个 aa 填充字节。
db(1);
pad(3, 0xaa);
db(2);
```

结果为：

```
01 aa aa aa 02
```

填充值实参可以省略，默认值为零：

```
// 使用默认填充值写出四个零字节。
pad(4);
```

pad_to(position, fill) 会持续写出填充字节，直到当前输出字节位置达到指定位置：

```
// 当前位置为 2；用 90 填充到位置 6，再写出 33。
db(0x11, 0x22);
pad_to(6, 0x90);
db(0x33);
```

输出：

```
11 22 90 90 90 90 33
```

指定位置不能位于当前输出位置之前。

align(boundary, fill) 会让输出位置前进到边界的下一个整数倍：

```
// 三个初始字节之后，用 cc 补齐到 8 字节边界。
db(0x11, 0x22, 0x33);
align(8, 0xcc);
db(0x44);
```

这会在 **44** 之前写出五个 **cc** 字节。填充值实参默认也是零。

对齐边界必须是非零的二次幂。1、2、4、16 和 4096 等值有效，3 或 24 等值会被拒绝。

应根据意图选择操作：

需求	使用
已知大小的未初始化空间	reserve 或 rb/rw/rd/rp/rq/rt/rdq/rqq/rdqq
精确数量的填充字节	pad
到达已知输出位置	pad_to
到达下一个对齐位置	align

声明二进制结构体

结构体为一组顺序排列的字段指定名称和类型：

```
// 自然布局会按字段类型的对齐要求安排偏移。
struct NaturalHeader {
    tag: u8
    size: u32
}
```

自然对齐。**tag** 从偏移 0 开始。**u32** 要 4 字节对齐，所以 **size** 从偏移 4 开始。总共 8 字节：

```
偏移 0: tag
偏移 1: 填充
偏移 2: 填充
偏移 3: 填充
偏移 4: size
偏移 5: size
偏移 6: size
偏移 7: size
```

自然布局会对齐每个字段，并把最终大小向上取整到结构体的对齐要求。它适合布局应遵循各字段对齐要求的内存记录。

如果需要精确的文件布局，应声明紧凑结构体：

```
// packed 禁止在字段之间和结构体末尾自动插入填充。
packed struct FileHeader {
    tag: u8
    size: u32
}
```

FileHeader 占五个字节。**size** 从偏移 1 开始，紧接在 **tag** 之后，末尾也没有填充。

紧凑布局会移除内部和尾部填充，但类型仍保留其最大字段的对齐属性。紧凑复合类型嵌套在自然布局复合类型中时，外层仍会按照这个对齐属性放置该字段。

文件头、协议记录、指令元数据和其他由外部规范规定的字节布局，通常应使用紧凑布局。运行时内存记录通常应使用自然布局。

字段默认值与结构体字面量

结构体的整数字段可以提供编译期默认值：

```
// magic 和 flags 提供默认值，size 由每个实例显式指定。
packed struct Header {
    magic: u16 = 0x5a4d
    flags: u16 = 1
    size: u32
}

// 字面量只覆盖 size，其余字段沿用声明中的默认值。
const header: Header = Header {
    size: 0x40
}

// 通过普通字段访问读取需要写出的成员。
emit.u16(header.magic);
emit.u32(header.size);
```

这个字面量提供了 **size**，并对 **magic** 和 **flags** 使用默认值。字面量中的字段按名称匹配，而不是按源码中的排列顺序匹配。

每个省略的字段都必须具有默认值。未知字段、重复字段和值类型不正确的字段都会导致源代码错误。

联合体字段不能声明默认值。联合体值必须始终显式选择且只选择一个当前字段。

字段可以使用普通的字段访问语法读取，如示例中的两个写出调用所示。

复合值存在于汇编期间。仅仅声明复合值并不会自动把它写入输出内容。

测量布局

sizeof(Type) 返回二进制类型的完整大小：

```
// 自然布局需要为 u32 字段和结构体末尾保留对齐空间。
struct NaturalHeader {
    tag: u8
    size: u32
}
```

```
// 紧凑布局保持字段连续排列。
packed struct PackedHeader {
    tag: u8
    size: u32
}

// 同时验证两种布局的总大小和关键字段偏移。
assert(sizeof(NaturalHeader) == 8);
assert(sizeof(PackedHeader) == 5);
assert(offset_of(NaturalHeader, tag) == 0);
assert(offset_of(NaturalHeader, size) == 4);
assert(offset_of(PackedHeader, size) == 1);
```

offset_of(Type, field) 返回字段偏移。这两个操作都是编译期表达式，可以用在指令、写出字段、断言和其他布局计算：

```
// 选择 64 位 x86 指令编码。
x86.use64();

// 两个 64 位寄存器槽位组成连续的保存区。
packed struct SaveArea {
    rax: u64
    rcx: u64
}

// 根据结构体大小调整栈指针，避免硬编码保存区字节数。
sub rsp, sizeof(SaveArea)
```

使用符号化的布局计算，可以避免新增字段或更改类型后，源代码其他位置仍残留过期的硬编码大小。

在 x86-64 栈上使用结构体布局

复合类型是汇编期的布局描述。x86 没有可以把整个结构体值一次性压栈的指令。正确做法是先在栈上分配记录空间，再把 **offset_of** 直接用于普通内存操作数：

```
x86.use64();

struct StackFrame {
    tag: u8,
    value: u32,
    tail: u16,
}

assert(sizeof(StackFrame) == 12);
assert(offset_of(StackFrame, value) == 4);
assert(offset_of(StackFrame, tail) == 8);

sub rsp, sizeof(StackFrame)
mov dword [rsp + offset_of(StackFrame, value)], 0x44332211
mov eax, [rsp + offset_of(StackFrame, value)]
mov word [rsp + offset_of(StackFrame, tail)], 0x6655
add rsp, sizeof(StackFrame)
```

紧凑布局与嵌套布局使用相同写法。嵌套字段可以直接写成 **[rsp + offset_of(NestedFrame, point.y)]**。如果这段代码还要调用 Windows x64 函数，必须另外满足调用约定的栈对齐与影子空间要求；**sizeof(StackFrame)** 只描述结构体记录本身。

在 PE64 程序中使用结构体栈帧时，同样要同时满足 Windows x64 调用约定；结构体大小 只负责描述局部记录的布局，不会自动补齐调用边界需要的栈空间。

打包并写出结构体值

pack(value) 会把复合值转换为 **bytes** 值：

```
// 两个小端 u16 字段共同组成四个连续字节。
packed struct Header {
    magic: u16 = 0x4241
    tail: u16
}

const header: Header = Header {
```

```

    tail: 0x4443
}

// 先打包以便比较，再把同一字节序列写入输出。
const encoded: bytes = pack(header)
assert(encoded == b"ABCD");
emit.bytes(encoded);

```

emit.struct(value) 会直接完成打包和写出：

```

// 自然布局会在 tag 与 size 之间加入三个零填充字节。
struct NaturalHeader {
    tag: u8 = 0x41
    size: u32 = 0x11223344
}

// 空字面量使用所有字段默认值，并立即写出完整结构体布局。
const header: NaturalHeader = NaturalHeader {}
emit.struct(header);

```

这会写出八个字节，其中自然布局产生的三个填充字节都是零：

```
41 00 00 00 44 33 22 11
```

如果还需要比较、转换、把字节序列保存在集合中，或者把它传给另一个函数，应使用 **pack**。如果只需要立即写出该值，应使用 **emit.struct**。

联合体与当前字段

联合会会让多个不同类型的字段重叠在偏移 0：

```

// Point 的两个坐标以紧凑形式连续存放。
packed struct Point {
    x: u16
    y: u16
}

// raw 与 point 共享同一段四字节存储。
union ValueBits {
    raw: u32
    point: Point
}

// 此字面量选择 point 作为联合体的当前字段。
const coordinates: ValueBits = ValueBits {
    point: Point {
        x: 0x1122,
        y: 0x3344
    }
}

```

联合体的每个字段都从偏移 0 开始。自然布局的联合体以最大字段大小为基础，再按最大的字段对齐要求向上取整。**packed** 联合体则直接使用最大字段的精确大小。

coordinates 字面量只选择一个当前字段。联合体字面量既不能省略所有字段，也不能同时初始化多个字段。联合体字段声明不能提供默认值，因为默认值不能表示一次显式的当前字段选择。

联合体还可以嵌套在结构体中：

```

// Point 描述联合体的一种四字节解释方式。
packed struct Point {
    x: u16
    y: u16
}

union ValueBits {
    raw: u32
    point: Point
}

// 紧凑记录把种类字节与联合体内容连续排列。
packed struct Record {
    kind: u8

```

```

    value: ValueBits
}

// 此实例选择 raw 作为嵌套联合体的当前字段。
const record: Record = Record {
  kind: 1,
  value: ValueBits {
    raw: 0xaabbccdd
  }
}

// 嵌套字段路径会累计外层和内层字段的偏移。
assert(offset_of(Record, value) == 1);
assert(offset_of(Record, value.point.y) == 3);
emit.struct(record);

```

如示例中的两个断言所示，`offset_of` 支持嵌套字段路径。

第二个结果把 `value` 在 `Record` 中的偏移，与 `y` 在 `Point` 中的偏移相加。

选择布局方式

以下情况适合直接写出整数和字节：

- 布局只有少数字段；
- 字段只写出一次，之后不再需要名称；
- 源代码与外部格式强绑定，显式调用比类型声明更清晰。

以下情况适合使用紧凑结构体和联合体：

- 精确的二进制记录会被重复使用；
- 字段名称可以提高可读性；
- 其他计算应由 `sizeof` 和 `offset_of` 驱动；
- 值需要默认值、嵌套、比较或转换为 `bytes`。

以下情况适合使用自然布局结构体：

- 记录表示对齐的内存，而不是序列化后的文件布局；
- 填充是有意保留的，并且应遵循字段对齐要求。

外部规范定义的布局用 `sizeof` 和 `offset_of` 验证。断言把格式假设变成可执行检查，后续修改也更安全。

下一章介绍模块和文件：include/import 组织源代码，编译期读取 JSON、TOML 等外部数据。

返回语言指南

第 10 章：模块与文件

汇编项目很快就会超过单个源文件的规模。指令辅助函数、二进制记录定义、生成的表格、配置数据和嵌入数据，通常由不同的人维护，变更原因也各异。

XIRASM 提供两种独立的文件加载模型：

- `include` 和 `import` 加载 XIRASM 源文件；
- `fs`、`json` 和 `toml` 函数加载 Meta 代码使用的数据。

区分这两种角色可以让项目结构更清晰：源文件提供声明或执行输出操作；数据文件被读取后变成源码可以检查、转换和写出的值。

> 当前 I/O 状态： `include/io/` 仍在开发中，尚不是稳定的标准库接口，用户程序不应把它当作稳定依赖。本章介绍的编译期 `fs.*` API 仍用于读取汇编输入；运行时 I/O 目前仍需要用户自己编写 ISA/ABI 例程，直到 `include/io/` 明确稳定为止。

导入源模块

`import(path)` 对给定的源文件至多求值一次：

```
// 同一个模块导入两次，模块内容也只会求值一次。
import("support.inc");
import("support.inc");

emit_word(0x1234);
```

假设 `support.inc` 包含：

```
fn emit_word(value: u64) {
    emit.u16(value);
}
```

两次导入指向同一个已解析文件，因此函数只声明一次。示例输出：

```
34 12
```

定义可复用函数、常量、结构体或其他名字的文件应使用 `import`。即使通过多条依赖路径引入同一模块，其声明和输出也不会重复。

`import` 语句必须位于源文件顶层，不可放置于可能按条件执行的块中：

```
if target.bits == 64 {
    import("x64-support.inc");
}
```

应在顶层选择要导入的模块；如果行为取决于目标，就把目标判断写在模块内部。

内联包含源文件

`include(path)` 每执行到一次就对目标源文件重新求值：

```
// 同一个包含文件执行两次，因此其中的输出操作也执行两次。
include("inline.inc");
include("inline.inc");
```

如果 `inline.inc` 包含：

```
emit.u8(0xaa);
```

输出为：

```
aa aa
```

仅在确实需要重复执行时使用 `include`。典型用途包括：

- 在当前输出位置插入生成的数据；
- 共享一段短的写出语句序列；
- 从计算得出的路径选取源文件片段；
- 多次应用同一个源文件模板。

由于包含操作重复执行，多次包含一个声明了相同函数、常量或类型的文件可能导致重名错误。这类文件通常是模块，应通过 **import** 加载。

源文件路径解析

相对路径以包含 **include** 或 **import** 语句的文件所在目录为基准进行解析。

对于以下项目：

```
project/
  main.asm
  tables/
    records.inc
  shared/
    constants.inc
```

tables/records.inc 可以加载相邻文件：

```
import("shared/constants.inc");
```

该路径相对于 **records.inc**，而非进程工作目录或入口源文件。

如果在当前源文件旁未找到相对路径对应的文件，XIRASM 会依次检查项目的 **include** 目录及其安装目录下的 **include** 目录。这使项目模块能够覆盖或补充已安装的库，而无需在源码中写入本机绝对路径。

绝对路径也可使用，但可移植的项目应优先使用相对于源文件的路径或已配置的 **include** 根目录。

源文件加载不允许形成循环。一个文件正在求值期间，不得直接或间接地再次包含或导入自身。

选择 **import** 还是 **include**

应根据执行语义选择，而非文件扩展名：

需求	使用
仅声明一次可复用名字	import
共享 Meta 函数或类型库	import
在当前输出位置执行源文件片段	include
重复执行同一源文件	include
避免依赖链中的重复声明	import

项目约定的常见做法：

- 模块定义可复用的名字和类型；
- 包含的片段执行与输出位置相关的工作；
- 入口源文件选择目标并组装最终输出。

检查与读取数据文件

fs.exists(path) 检查数据文件是否能按当前路径规则找到：

```
// 确认文件存在，再把它的全部字节写入输出。
assert(fs.exists("payload.bin"));
emit.bytes(fs.read_bytes("payload.bin"));
```

数据路径使用与源文件加载相同的相对路径解析模型。当这段代码移入模块时，`payload.bin` 会相对于该模块所在文件解析。

`fs.read_text(path)` 将整个文件读为 `string`：

```
const banner: string = fs.read_text("banner.txt")
assert(contains(banner, "XIRASM"));
emit.bytes(banner);
```

如果 `banner.txt` 包含：

```
XIRASM
```

按原样写出文件中的字节，不会自动追加终止零字节。

`fs.read_bytes(path)` 将整个文件读为 `bytes`。图片、编码后的表格、预构建的记录及其他二进制数据均适用。

当字节仅用于输出时，`emit.file(path)` 可避免创建中间绑定。`emit.file(path, offset, count)` 写出精确的范围。这两种形式均使用与 `fs.read_bytes` 相同的相对路径解析器和边界检查，但不能在 `late_layout` 和 `defer` 中使用。

读取字节范围

三参数形式的 `fs.read_bytes` 读取带边界的范围：

```
const middle: bytes = fs.read_bytes("payload.bin", 1, 2)
assert(len(middle) == 2);
emit.bytes(middle);
```

第二个参数是基于零的文件偏移，第三个参数是返回的字节数。

如果 `payload.bin` 包含：

```
10 20 30 40
```

示例输出：

```
20 30
```

所请求的范围必须完整存在。XIRASM 会报告越界读取错误，而非静默截断。

加载 JSON

`json.file(path)` 一次性完成 JSON 文件的读取和解析：

```
const config: map = json.file("config.json")
const values: list = map.get(config, "values")

assert(map.get(config, "enabled"));
emit.bytes(map.get(config, "name"));
emit.u8(map.get(config, "bits"));
emit.u8(list.get(values, 0));
emit.u8(list.get(values, 1));
```

对于以下输入：

```
{
  "name": "XR",
  "bits": 64,
  "enabled": true,
  "values": [1, 2]
}
```

输出为：

```
58 52 40 01 02
```

`json.parse(value)` 解析已包含在 `string` 或 `bytes` 值中的 JSON：

```
const raw: string = fs.read_text("config.json")
const config: map = json.parse(raw)
```

```
emit.u8(map.get(config, "bits"));
```

JSON 对象转换为 map，数组转换为 list，字符串和布尔值保持相应的 Meta 类型，非负整数转换为整数值。JSON 的 **null** 转换为 **void**。

浮点数和负整数不在当前 Meta 数据模型的范围。重复的键和格式错误的 JSON 将被拒绝。

加载 TOML

toml.file(path) 为 TOML 提供同样的便捷入口：

```
const config: map = toml.file("project.toml")
const target: map = map.get(config, "target")

emit.bytes(map.get(config, "name"));
emit.u8(map.get(target, "bits"));
```

对于以下输入：

```
name = "XR"

[target]
bits = 64
```

输出为：

```
58 52 40
```

toml.parse(value) 从 **string** 或 **bytes** 值中解析 TOML：

```
const raw: string = fs.read_text("project.toml")
const config: map = toml.parse(raw)
const target: map = map.get(config, "target")

assert(map.get(target, "bits") == 64);
emit.u8(0x40);
```

TOML 表转换为 map，数组转换为 list，字符串、布尔值和非负整数转换为相应的 Meta 值。Meta 数据转换目前不接受浮点数和时间戳值。

区分配置与二进制数据

结构化配置和原始二进制数据解决不同的问题。

使用 JSON 或 TOML 的场景：

- 字段需通过名称访问；
- 输入预期由人工编辑；
- 同一输入驱动多个生成值；
- 需进行校验和条件生成。

使用 **fs.read_bytes** 的场景：

- 文件内容已是所需的二进制表示；
- 需保持字节级的精确性；
- 仅需有限的字节范围；
- 解析不会带来有用的结构。

当文本本身就是数据，或需显式应用某个解析器时，使用 **fs.read_text**。

实用的模块组织方式

中等规模的项目可采用以下结构：

```
project/  
  main.asm  
  include/  
    records.inc  
    encoding.inc  
  data/  
    config.toml  
    payload.bin
```

main.asm 导入可复用的定义：

```
import("records.inc");  
import("encoding.inc");
```

这些模块可以相对于自身读取数据，或通过显式路径（如 `../data/config.toml`）读取。

应保持模块行为的可读性：

- 声明库优先使用 **import**;
- **include** 仅用于确实需要重复执行的工作;
- 文件路径相对于所属源文件书写;
- 用 **assert** 和 **fs.exists** 验证必要数据;
- 结构化文件仅解析一次，复用得到的 `map` 或 `list`;
- 大型文件仅需部分内容时，使用有界的二进制读取。

下一章介绍输出区域与虚拟数据：XIRASM 如何分离逻辑地址、物理文件字节、预留空间和临时布局区域。

[返回目录](#)

第 11 章：输出区域与虚拟数据

汇编器经常要同时回答两个问题：这个值处在什么逻辑地址？它的字节会不会出现在输出文件中，如果会，又位于哪个文件偏移？

简单的 flat binary 往往让两者都从零开始并同步前进。结构化二进制则经常把它们拆开：某块内容可以加载到高地址，却仍然靠近文件开头；预留内存可以扩大地址范围而不增加文件字节；临时生成的数据也可能完全不进入最终文件。

本章说明输出区域如何表达这些情况，以及逻辑地址、文件偏移、物理文件字节和临时布局区域之间的关系。

XIRASM 用输出区域来建模这些坐标。

逻辑坐标与物理坐标

每个普通区域有四条属性：

- origin：标号地址的基准；
- file offset：区域字节在文件中的物理起点；
- logical size：包含预留空间在内的逻辑地址范围；
- file size：最终实际进入文件的字节数。

这四项有意保持独立。改变 origin 不会插入文件填充；改变 file offset 也不会改变标号值。

常用查询：

查询	含义
<code>region_base()</code>	当前区域的 origin
<code>here()</code>	当前逻辑地址
<code>file_offset()</code>	当前实际文件位置
<code>file_cursor_real()</code>	当前已物理化的文件光标
<code>file_cursor_potential()</code>	如果当前预留尾部都物理化时的文件位置
<code>tail_reserve_size()</code>	当前区域尾部尚未物理化的预留字节数

这些光标查询主要用于自定义布局和可复用的格式辅助函数。普通代码通常只需要标号和 `here()`。

设置 Origin

`origin(address)` 设定当前区域的地址起点：

```
// 把当前输出区域的逻辑起点设为 0x4000。
origin(0x4000);

start:
// 在起始地址处写出一个字节。
emit.u8(0xaa);

// 检查区域起点、标号地址、当前位置和文件位置。
assert(region_base() == 0x4000);
assert(label_addr("start") == 0x4000);
assert(here() == 0x4001);
assert(file_offset() == 1);
```

输出一字节：

```
aa
```

`origin` 改变的是标号地址基准，不是文件中的物理位置。简单 `flat` 区域通常先设 `origin`，再定义标号或写出内容。之后再次调用 `origin` 会更新整个当前区域的基准，而不是创建新区域。

创建显式区域

`region.begin(name, origin, file_offset)` 同时设逻辑地址和文件偏移：

```
// 头部的逻辑地址从 0x1000 开始，文件位置从 0 开始。
region.begin("header", 0x1000, 0);

header:
emit.bytes(b"HD");

// 数据使用独立的逻辑地址，并从文件偏移 0x10 开始。
region.begin("payload", 0x2000, 0x10);

payload:
emit.bytes(b"DATA");

// 两个标号分别使用各自输出区域的起始地址。
assert(label_addr("header") == 0x1000);
assert(label_addr("payload") == 0x2000);
```

`header` 占文件偏移 0 和 1，`payload` 从 0x10 开始。中间未写的部分在 `flat` 输出里补零：

```
48 44 00 00 00 00 00 00 00 00 00 00 00 00 00 00
44 41 54 41
```

开始另一个区域会切换当前输出区域。区域之间不是词法嵌套关系，也没有对应的 `end` 语句。

区域名描述源码层面的布局意图；它本身不会为任何可执行文件或目标文件格式创建 `section` 记录。那些记录属于格式接口的职责。

`region.begin` 也不是“插入到已有字节中间”的命令。它只是把后续输出切到一个由调用者明确指定逻辑 `origin` 和文件偏移的区域。区域之间的顺序、文件偏移是否重叠、是否留有物理空洞，都由调用者的布局模型负责。构造 PE/ELF/COFF 这类格式时，格式接口会替你维护这些关系；手写自定义格式时，必须自己把文件偏移、逻辑地址和最终回填字段分开记录。

当逻辑 `origin` 和物理文件偏移都已经明确时，使用 `region.begin`。如果只是按顺序构造文件，`output.section` 和 `output.org` 通常更稳妥，因为它们会从当前区域推导下一段的文件偏移。

实际光标与潜在光标

写已初始化字节时，两个光标同步推进：

```
// 写出一个实际存在于文件中的字节。
emit.u8(0xaa);

// 实际位置和潜在位置都前进到 1，尾部没有预留空间。
assert(file_cursor_real() == 1);
assert(file_cursor_potential() == 1);
assert(tail_reserve_size() == 0);
```

`reserve` 会推进潜在光标和逻辑地址，但不会立刻提交物理文件字节：

```
// 先写出一个字节，再预留三个尚未实际写入的字节。
emit.u8(0xaa);
reserve(3);

// 逻辑地址已经前进四字节，实际文件仍只有一个字节。
assert(here() == 4);
assert(file_cursor_real() == 1);
assert(file_cursor_potential() == 4);
assert(tail_reserve_size() == 3);
```

预留后面出现初始化数据时，预留范围变成文件中间的间隙：

```
// 后续字节使前面的预留范围变成文件中的实际间隙。
emit.u8(0xaa);
```

```
reserve(3);
emit.u8(0xbb);

// 中间间隙已经物理化，两个文件光标重新一致。
assert(file_cursor_real() == 5);
assert(file_cursor_potential() == 5);
assert(tail_reserve_size() == 0);
```

文件内容：

```
aa 00 00 00 bb
```

这种延迟物理化的语义，让一次 **reserve** 既能表示文件中间的零填充间隙，也能表示不占文件空间的尾部预留。

裁掉预留尾部

output.section(name, origin) 从当前实际文件光标处开始新区域。

```
// 第一个区域包含一个实际字节和三个尾部预留字节。
emit.u8(0x41);
reserve(3);

// 新区域紧接最后一个实际字节，尾部预留空间不进入文件。
output.section("next", 0x2000);
emit.u8(0x42);
```

尾部预留会增大第一区域的逻辑大小，但新区域从最后一个已物理化字节之后开始。输出：

```
41 42
```

适用于运行时需要地址范围但文件末尾不占空间的场景。

output.section 只丢弃尚未物理化的尾部预留，文件中间的间隙不受影响。

保留潜在间隙

output.org(name, origin) 从当前潜在光标处开新区域。

```
// 第一个区域的潜在长度为四字节。
emit.u8(0x41);
reserve(3);

// 新区域从潜在位置开始，因此保留前面的三字节间隔。
output.org("next", 0x2000);
emit.u8(0x42);
```

预留之后写入数据，预留范围保留在文件中：

```
41 00 00 00 42
```

区别：

操作	下一个文件偏移起点
output.section	实际光标，裁尾部预留
output.org	潜在光标，保留预留范围

两个操作都会独立选择新的逻辑 origin；这个选择与文件偏移无关。

对齐区域文件大小

region.file_align(alignment) 对齐当前区域最终的物理文件大小。

```
// 第一个区域从逻辑地址 0x1000、文件偏移 0 开始。
region.begin("first", 0x1000, 0);

// 三个实际字节之后还有十三个尾部预留字节。
emit.bytes(b"ABC");
reserve(13);
assert(here() == 0x1010);
```



```
assert(file_cursor_real() == 3);
assert(file_cursor_potential() == 16);

// 去除尾部预留空间，再把实际文件大小向上对齐到八字节。
region.file_align(8);

// 第二个区域从已经对齐的文件偏移 8 开始。
region.begin("second", 0x2000, 8);
emit.u8(0x5a);
```

裁掉尾部预留后，三个物理字节按 8 字节边界对齐。第二区域起始文件偏移 8：

```
41 42 43 00 00 00 00 5a
```

文件对齐不会推进逻辑大小。本例中，第一区域仍然描述 16 个逻辑字节。

对齐值必须是非零的 2 的幂。调用 `region.file_align` 会关闭当前区域的物理输出；继续写字节前，需要先开始另一个区域。

它不同于 `align`：`align` 会推进逻辑位置，并在间隙物理化时写出填充字节；`region.file_align` 只把最终物理大小向上取整，不改变逻辑地址。

虚拟输出

虚拟区域是汇编期间的隔离输出区域。它的字节、标号和指令可以被读取或转换，但不会自动复制到最终 flat 输出中。

`virtual.begin(origin)` 在指定逻辑地址创建虚拟区域：

```
// 在逻辑地址 0x3000 创建一个临时虚拟区域。
virtual.begin(0x3000);

table:
emit.u32(0x11223344);
// 读取原值、逐位变换后，再写回同一位置。
store.u32(table, load.u32(table) ^ 0x01010101);
const encoded: bytes = load.bytes(table, 4)

virtual.end();

// 只有显式复制出的字节才会进入主输出。
emit.bytes(encoded);
```

临时区域初始内容：

```
44 33 22 11
```

变换后复制到主输出：

```
45 32 23 10
```

虚拟区域本身不贡献文件字节。

虚拟区域里可以正常写出、预留、对齐、定义标签、写 ISA 指令。`load.*` 和 `store.*` 对其字节同样生效。每个 `virtual.begin` 对应一个 `virtual.end`。

虚拟区域只能作为临时输出空间使用。它不能在里面启动主输出区域：`output.section` 和 `output.org` 必须等回到主输出后再调用。虚拟区域里的文件光标也不能当成最终文件存储位置来推导；需要最终文件偏移时，应把虚拟字节显式复制到真实区域，再在稳定阶段查询真实区域的事实。

使用当前地址作为虚拟 Origin

省略 `origin` 参数时，虚拟区域从外围区域的当前逻辑地址开始：

```
// 主输出从逻辑地址 0x4000 开始，并先写出一个字节。
origin(0x4000);
emit.u8(0xaa);

// 省略参数，使虚拟区域沿用当前逻辑地址 0x4001。
```

```
virtual.begin();

scratch:
emit.u16(0x1234);
const copied: bytes = load.bytes(scratch, 2)

virtual.end();

// 显式把临时生成的两个字节追加到主输出。
emit.bytes(copied);
```

scratch 逻辑地址 0x4001，但其虚拟字节不替换也不推进外围区域。**virtual.end** 后主输出从暂停处继续。最终文件：

```
aa 34 12
```

适用于编码需临时组装、且标签地址需与目标位置共用同一基准的场景。

稳定的区域事实

普通写出阶段只有实时光标可用。区域的最终文件偏移、物理大小和逻辑大小要等布局稳定后才确定。以下查询只在收尾处理阶段可用：

查询	对包含指定地址的区域返回
<code>region_file_offset(address)</code>	区域在文件中的基偏移
<code>region_file_size(address)</code>	最终物理大小
<code>region_logical_size(address)</code>	最终逻辑大小

这些查询和实时光标故意分开。收尾处理可以用它们回填文件头，或断言自定义格式声明的大小与最终布局一致。下一章会介绍这个阶段及其限制。

选择合适的工具

只用 `origin`：

- 单个 flat 流需非零地址基准
- 文件位置和逻辑位置同步推进

output.section：

- 下一段应跟在已物理化字节之后
- 尾部预留应增加内存大小但不占文件空间

output.org：

- 下一段应从潜在位置继续
- 后续已初始化数据之前的预留空间必须留在文件中

region.begin：

- 逻辑 `origin` 和物理文件偏移都明确
- 自定义二进制布局或格式辅助函数

虚拟区域：

- 临时组装、测量、读取或转换数据
- 临时字节不得自动进入主输出

总结：

- 标号和 `here()` 描述逻辑地址
- 实际文件光标只反映已物理化的字节
- 潜在光标包含当前尾部预留
- 最终区域事实属于稳定后的收尾处理阶段

下一章介绍收尾处理：读取已稳定的布局信息、修补字节、计算校验和、验证映像，不改变布局。

[返回目录](#)

第 12 章：收尾处理

二进制文件中的许多字段在首次写入时无法获得最终值。例如：

- 稍后才声明的数据区大小；
- 另一区域的文件偏移；
- 预留区域的最终逻辑大小；
- 对已编码指令和已修补 fixup 的校验和；
- 由头文件后方的源码累加得到的计数。

XIRASM 通过显式的收尾阶段处理这些情况，而不是反复执行整个源文件直到数值碰巧收敛。

该阶段提供两种工具：

- **late_layout** 在映像封存前执行受限的布局变更步骤；
- **defer** 读取并修补稳定后的最终映像，但不改变布局。

大多数用户编写的回填逻辑都应放在 **defer** 中。

为回填预留字段

标准流程如下：

- 写入固定宽度的占位值；
- 正常写入数据；
- 在 **defer** 中修补占位字段。

```
// 先用两个字节为数据大小预留位置。
size_field:
emit.u16(0);

// 写出实际数据，并用前后标号确定它的长度。
payload:
emit.bytes(b"ABC");
payload_end:

defer {
    // 布局稳定后，把数据长度回填到预留字段。
    store.u16(size_field, payload_end - payload);
}
```

最终输出：

```
03 00 41 42 43
```

占位字段已经占据两个字节，收尾处理只修改这两个字节的值，不插入新字节，也不移动 payload。

顶层的收尾处理块可在其引用的标签之前声明：

```
defer {
    // 这些标号可以在收尾块之后定义，最终执行时会得到稳定地址。
    store.u16(size_field, payload_end - payload);
}

// 为数据长度预留两个字节。
size_field:
emit.u16(0);

// 随后写出实际数据。
payload:
emit.bytes(b"ABC");
payload_end:
```

稳定的最终映像可用时，这些标号已经解析完成。

读取最终映像

`load.u8`、`load.u16`、`load.u32` 和 `load.u64` 从已物理化的输出字节中读取小端整数。`load.bytes(address, count)` 返回字节范围。

这些读取操作可与存储和断言组合使用：

```
// 让本区域的逻辑地址从 0x4000 开始。
origin(0x4000);

// 预留两个 32 位文件头字段。
header:
emit.u32(0);
emit.u32(0);

// 写出正文和一个稍后替换的尾部标记。
body:
emit.bytes(b"ABCD");
tail:
emit.bytes(b"???");
image_end:

defer {
    // 回填正文到输出末尾的长度，以及正文相对区域起点的位置。
    store.u32(header, image_end - body);
    store.u32(header + 4, body - region_base());
    store.bytes(tail, b"OK!!");

    // 读取最终字节，确认回填结果符合预期。
    assert(load.u32(header) == 8);
    assert(load.u32(header + 4) == 8);
    assert(load.bytes(tail, 4) == b"OK!!");
}
```

完成后的字节：

```
08 00 00 00 08 00 00 00 41 42 43 44 4f 4b 21 21
```

`store.bytes` 接受字符串或 `bytes` 值。整数存储会拒绝超出目标宽度的值。

所有加载和存储都会做范围检查。收尾处理只能访问物理映像中实际存在的字节。位于已裁掉预留尾部中的逻辑地址不是可写占位字段，会被拒绝。

计算校验和

收尾处理可声明局部值、更新可变值并使用 `while`：

```
// 使用固定逻辑起点，便于按标号读取数据。
origin(0x5000);

// 先为 16 位校验和预留位置。
checksum:
emit.u16(0);

// 写出参与校验和计算的数据。
payload:
emit.bytes(b"ABCD");
payload_end:

defer {
    // 逐字节读取最终数据并累加。
    let cursor = payload
    let sum = 0

    while cursor < payload_end {
        sum = sum + load.u8(cursor)
        cursor = cursor + 1
    }

    // 回填校验和，并立即检查写入结果。
```

```
store.u16(checksum, sum);
assert(load.u16(checksum) == 266);
}
```

校验和为 266 (即 0x010a) , 输出:

```
0a 01 41 42 43 44
```

收尾处理中的循环使用与普通 Meta 循环相同的编译期迭代限制。for 目前不允许出现在收尾处理中; 逐字节扫描应使用有界的 while 循环。

使用稳定的区域信息

第 11 章介绍的最终区域查询可在 defer 中使用:

```
// 创建逻辑起点为 0x5000、文件起点为 0 的输出区域。
region.begin("payload", 0x5000, 0);

// 为区域的文件大小和逻辑大小预留字段。
file_size_field:
emit.u32(0);
logical_size_field:
emit.u32(0);

// 写出一个实际字节, 再追加三个只占逻辑空间的预留字节。
body:
emit.u8(0xaa);
reserve(3);

defer {
    // 布局稳定后查询正文所在区域, 并回填最终大小。
    store.u32(file_size_field, region_file_size(body));
    store.u32(logical_size_field, region_logical_size(body));

    // 文件大小不包含未实际写出的预留尾部, 逻辑大小包含它。
    assert(region_file_offset(body) == 0);
    assert(region_file_size(body) == 9);
    assert(region_logical_size(body) == 12);
}
```

预留尾部计入逻辑大小, 但不计入物理文件大小:

```
09 00 00 00 0c 00 00 00 aa
```

这些查询返回包含指定逻辑地址的区域信息:

- `region_file_offset(address)` 返回区域的物理基偏移;
- `region_file_size(address)` 返回最终物理大小;
- `region_logical_size(address)` 返回完整的地址空间大小。

它们依赖稳定布局, 因此不能在初始写出阶段当作实时光标查询使用。

调用值函数

返回值的 Meta 函数可用于纯粹的最终计算:

```
// 计算不小于 value 且满足指定对齐要求的数值。
fn align_up(value: u64, alignment: u64) -> u64 {
    return ((value + alignment - 1) / alignment) * alignment;
}

// 为对齐后的数据大小预留字段。
size_field:
emit.u32(0);

payload:
emit.bytes(b"ABC");
payload_end:

defer {
```

```
// 把三字节数据向上对齐到八字节，并回填计算结果。
store.u32(size_field, align_up(payload_end - payload, 8));
assert(load.u32(size_field) == 8);
}
```

最终输出：

```
08 00 00 00 41 42 43
```

该函数只做表达式运算，不自行写出或修补字节。

可复用的回填过程

过程可以注册收尾处理块并捕获其参数：

```
// 登记一个稍后写入 16 位整数的收尾块。
fn patch_u16(address: u64, value: u64) {
  defer {
    // address 和 value 保存的是调用过程函数时传入的值。
    store.u16(address, value);
  }
}

// 先写出占位字段，再登记对应的回填操作。
field:
emit.u16(0);

patch_u16(field, 0x1234);
```

调用发生在普通源码处理阶段。传入的参数值会冻结到登记的收尾处理块中，后者随后写出：

```
34 12
```

这种模式适合小型、带类型的回填辅助过程。过程作用域内的参数会按值冻结；顶层收尾处理表达式则可以一直保持符号状态，直到最终求值。

过程并不是在 **defer** 内部调用。过程调用发生得更早，它只是登记了稍后执行的收尾处理块。

收尾处理的控制流

defer 体内可包含：

- **const** 和 **let** 声明；
- 对局部 **let** 值的赋值；
- **if** 和 **else**；
- **while**；
- **store.u8**、**store.u16**、**store.u32**、**store.u64** 和 **store.bytes**；
- **assert**、**print**、**warn** 和 **err**。

上述语句中的表达式可使用普通的纯运算符和值函数、标签、**load.*** 以及稳定的区域信息。

每个收尾处理块拥有独立的局部作用域。在一个块中声明的局部变量在另一个块中不可见。

以下操作因会改变布局而被拒绝：

```
defer {
  emit.u8(0x22);
}
```

相同的限制适用于 ISA 指令、标签、区域切换、对齐、预留、嵌套的收尾处理块、函数声明和源文件加载。

收尾处理的执行顺序

defer 块按注册顺序执行：

```
// 先写出一个稍后会被连续修改的字节。
emit.u8(0);

defer {
    // 第一个收尾块把初始值改为 1。
    store.u8(0, 1);
}

defer {
    // 第二个收尾块能看到前一次修改，因此最终写入 2。
    store.u8(0, load.u8(0) + 1);
}
```

第二个块可观察到第一个块的修补结果，输出：

02

应有意识地使用这个顺序。当两个收尾处理块修改相同字节时，后执行的块会看到前一个块的结果。

收尾处理块在以下步骤之后执行：

- 普通源码处理；
- 指令编码；
- 后期布局；
- fixup 解析；
- 最终布局和字节物理化；
- fixup 修补。

因此，它们看到的是将被实际写入的精确映像，包括已编码的指令和已解析的引用。

必须延后运行的布局工作

有时源文件必须等主源码完成内容登记后，才创建真实字节或放置真实区域。这正是 **late_layout** 的职责：

```
// 正常源码先写出第一个字节。
emit.u8(0x10);

late_layout {
    // 后期布局块按照登记顺序追加实际字节。
    emit.u8(0x20);
}

late_layout {
    emit.u8(0x30);
}
```

late_layout 块按注册顺序执行一次，从默认输出区域的末尾开始。示例输出：

10 20 30

后期布局会在 fixup 解析和最终布局之前完成。它写出的字节会参与最终布局和物理化。

默认行为确实是从当前默认输出尾部继续写。但 **late_layout** 不是“只能追加到整个文件末尾”的 API：块体允许使用输出区域 API，因此可以在块中 **region.begin(...)**，把晚生成的表、字符串池或重定位记录放到调用者明确选择的真实文件偏移处。

关键边界是：**late_layout** 创建的是尚未封存的布局内容，不是在最终映像中随机插入字节。若只写 **emit.***，它就沿默认输出尾部继续；若要把字节放入某个自定义区域，必须显式开始那个区域，并保证文件偏移、逻辑地址、大小字段和后续回填一致。

典型的直接构造流程是：

```
// 普通源码先写出固定头字段和主体。
```



```

table_offset_field:
emit.u32(0);
emit.bytes(b"HDR");

// 先在虚拟区域里组装并测量晚生成表。
const table_origin: u64 = 0x8000
virtual.begin(table_origin);
table:
emit.bytes(b"TAB");
const table_bytes: bytes = load.bytes(table, 3)
const table_size: u64 = here() - table
virtual.end();

// 主输出已经写完固定头和主体，记录表最终要落到的文件偏移。
const table_foa: u64 = file_cursor_real()

late_layout {
    // 在后期布局阶段打开真实文件区域，并把虚拟字节复制进去。
    region.begin("late-table", table_origin, table_foa);
    emit.bytes(table_bytes);
}

defer {
    // 稳定后再回填并验证头字段。
    store.u32(table_offset_field, table_foa);
    assert(load.u32(table_offset_field) == table_foa);
    assert(table_size == 3);
}

```

这个模式不只适用于 flat binary。COFF/ELF 之类的表也可以先用虚拟区域生成，再在 **late_layout** 中放进真实文件区域；只是标准 PE/COFF/ELF 通常应优先使用第 14 章的格式接口，让它维护 section、segment、表项和回填关系。

后期布局可物理化预留尾部

由于 **late_layout** 仍然会改变布局，追加的已初始化字节可能把先前的预留尾部转化为文件中间的间隙：

```

// 写出一个字节，并在逻辑地址中预留三个字节。
emit.u8(0xaa);
reserve(3);

late_layout {
    // 在预留空间之后追加实际字节，使中间空隙进入输出文件。
    emit.u8(0xbb);
}

```

最终输出：

```
aa 00 00 00 bb
```

只有预留范围确实应该变成物理文件字节时才使用这种行为。如果尾部应该保持 file-free，应在登记后续已初始化输出之前切换到另一个区域。

组合后期布局与最终回填

收尾处理块可看到后期布局期间追加的字节：

```

// 为整个区域的最终逻辑大小预留字段。
size_field:
emit.u32(0);

// 正常源代码先写出前两个数据字节。
emit.bytes(b"AB");

late_layout {
    // 后期布局再追加两个会参与最终大小计算的字节。
    emit.bytes(b"CD");
}

defer {
    // 根据稳定后的区域大小回填字段，并检查实际文件大小。
    store.u32(size_field, region_logical_size(size_field));
}

```

```
    assert(region_file_size(size_field) == 8);  
}
```

稳定后的区域包含四字节字段加四字节数据：

08 00 00 00 41 42 43 44

这正是两者的职责划分：

- **late_layout** 创建必须参与布局的字节；
- **defer** 观察最终布局并修补已有字段。

后期布局的限制

late_layout 比普通源码的范围窄。它接受布局和输出 API 调用、诊断和 **if** 选择。它可以：

- 写出整数、字节和结构体；
- 预留、填充和对齐；
- 切换或对齐输出区域；
- 打开和关闭虚拟区域；
- 存储到既有输出字节中。

这里的“存储到既有输出字节”仍然要求目标字节已经存在；它不是创建新空间的替代品。需要新增字节、预留空间、对齐或切换区域时，必须使用 **late_layout** 允许的布局 API 明确创建。

它不能声明标号、写出 ISA 指令文本、声明局部值、循环、定义函数、加载源模块，或注册嵌套的后期/收尾处理块。

后期布局只执行一次。它不是隐式多遍机制，不应用来反复让不稳定的值收敛。

选择正确的阶段

字节能够按正常源码顺序写出时，就使用普通源码。

使用 **late_layout** 的场景：

- 必须在主源码之后创建真实字节或真实区域；
- 晚生成表需要进入调用者明确选择的文件偏移；
- 这些字节必须影响最终的偏移和大小；
- 受限的一次性后期布局步骤已足够。

使用 **defer** 的场景：

- 固定宽度的占位字段需要最终值；
- 校验和或验证需要完整的字节映像；
- 需要最终的区域大小或偏移；
- 必须修补现有字节且不改布局。

切勿用 **defer** 创建缺失的空间。应在收尾处理之前预留或写出所需存储，然后只修补这个既有范围。

下一章进入第三部分，介绍 flat 和自定义二进制文件：将标签、结构体、区域、后期布局和收尾处理组合为完整的文件格式。

第三部分：构建程序

[返回目录](#)

第 13 章：Flat Binary 与自定义文件格式

XIRASM 默认写出 flat binary。输出文件只包含指令、数据声明、填充和输出区域最终物理化的字节；除非源码显式创建，否则不会自动带上操作系统文件头。

这种直接模型适合：

- 引导扇区、固件映像
- ROM 表、嵌入式资源
- 协议消息、测试数据
- 紧凑的资源容器
- 私有二进制格式
- 被其他程序加载的小段可执行代码

flat binary 仍然可以有丰富的内部结构。标号描述逻辑位置，结构体描述记录，区域分离逻辑坐标和物理坐标，收尾处理根据完整映像推导字段值。

原始机器码

ISA 指令就是 flat 文件的全部内容。

```
// 选择 64 位 x86 指令编码，输出文件只包含下面两条指令的机器码。
x86.use64();

entry:
// 把数值 42 写入 eax，然后返回给调用者。
mov eax, 42
ret
```

输出：

```
b8 2a 00 00 00 c3
```

这里没有可执行文件头、入口点字段或加载器元数据。**entry** 只是汇编器标号，不是操作系统入口点。加载这些字节的程序必须已经知道目标 ISA、指令模式、加载地址和调用约定。

组合文件头与 Payload

```
// 先写出四字节文件标识，再写出两个小端顺序的 16 位头字段。
emit.bytes(b"RAW1");
emit.u16(1);
emit.u16(0);

payload:
// 标号记录数据的逻辑起点，随后写出三个数据字节。
emit.bytes(b"ABC");
```

输出：

```
52 41 57 31 01 00 00 00 41 42 43
```

前四字节是文件签名，第一个 **u16** 存版本号，第二个 **u16** 目前保留，剩下的字节构成 payload。这里不需要特殊的文件格式声明；源文件顺序就是文件顺序。

用 Struct 描述记录

当二进制记录需要命名字段、且字段之间没有隐式对齐间隙时，使用 **packed struct**。

```
// 紧凑结构体按声明顺序连续存放两个 16 位字段。
packed struct ChunkHeader {
    kind: u16
```

```

    size: u16
}

// 写出记录头，随后紧接着写出三个字节的记录内容。
emit.struct(ChunkHeader {
    kind: 1,
    size: 3,
});
emit.bytes(b"ABC");

```

输出：

```
01 00 03 00 41 42 43
```

结构体让字段名和宽度留在源码里，`emit.struct` 会按 `packed` 表示写出。只有当文件格式本身要求同样的对齐和尾部填充时，才使用普通 `struct`。

重复的写出逻辑封装成函数：

```

// 每次调用都按 “一个字节的标签、四个字节的数值” 写出一条记录。
fn emit_record(tag: u8, value: u32) {
    emit.u8(tag);
    emit.u32(value);
}

// 两条记录按照调用顺序连续写入文件。
emit_record(1, 0x11223344);
emit_record(2, 0xaabbccdd);

```

输出：

```
01 44 33 22 11 02 dd cc bb aa
```

过程有助于保持自定义写出逻辑一致，但最终文件仍由普通源码顺序决定。

推导文件头字段

不要手工维护偏移、大小和校验和。先写出固定宽度占位字段，给相关数据打上标号，再在 `defer` 中推导最终值：

```

// 让本区域的逻辑地址从零开始，便于用标号差表示文件内距离。
origin(0);

magic:
emit.bytes(b"XIF1");
// 先为总大小、数据偏移和校验和各写出一个四字节占位字段。
size_field:
emit.u32(0);
payload_offset_field:
emit.u32(0);
checksum_field:
emit.u32(0);

payload:
emit.bytes(b"OK!!");
payload_end:

defer {
    // 布局稳定后，根据标号回填大小、偏移和前两个数据字节的校验和。
    store.u32(size_field, payload_end - magic);
    store.u32(payload_offset_field, payload - magic);
    store.u32(checksum_field, load.u8(payload) + load.u8(payload + 1));

    // 检查最终文件中的标识和三个回填字段。
    assert(load.bytes(magic, 4) == b"XIF1");
    assert(load.u32(size_field) == 20);
    assert(load.u32(payload_offset_field) == 16);
    assert(load.u32(checksum_field) == 0x9a);
}

```

输出：

```
58 49 46 31 14 00 00 00 10 00 00 00 9a 00 00 00 4f 4b 21 21
```

这个示例没有硬编码总大小或 payload 偏移。文件头或 payload 改变时，收尾处理会把新值写回既有字段。由于本例使用一个从 origin 0 开始的紧凑区域，**magic** 起算的逻辑距离同时也是文件内的 payload 相对偏移。

明确区分要存入字段的坐标：

- 格式需要逻辑距离时，使用标号相减；
- 需要当前物理光标时，在写出阶段使用 `file_offset()`；
- 需要某个区域最终物理基址时，在 `defer` 中使用 `region_file_offset(address)`；
- 需要稳定后的最终大小时，使用 `region_file_size(address)` 和 `region_logical_size(address)`。

区分逻辑地址与文件偏移

文件可以连续存放字节，同时给这些字节分配完全不同的逻辑地址。

```
// 文件头位于文件偏移 0，但它的逻辑地址从 0x1000 开始。
region.begin("header", 0x1000, 0);
emit.bytes(b"HDR0");

// 数据紧接文件头写入文件，但逻辑地址改从 0x2000 开始。
output.section("payload", 0x2000);
payload:
emit.bytes(b"DATA");

defer {
    // payload 正好位于区域起点，因此可用它查询区域的起始文件偏移。
    assert(payload == 0x2000);
    assert(region_file_offset(payload) == 4);
}
```

物理文件仍然紧凑：

```
48 44 52 30 44 41 54 41
```

payload 从文件偏移 4 开始，但逻辑地址是 **0x2000**。固件、ROM、内存映像以及描述加载位置的格式里经常需要这种区分。

除非格式明确规定二者关系，否则不要从逻辑地址推导文件偏移。应分别查询或记录各自的坐标。

表示已初始化间隙和 File-Free 尾部

reserve 是否进入文件，取决于后面是否还有已初始化数据：

- 文件中间的间隙会物理化为零字节；
- 连续的预留尾部可以只增加逻辑大小，不出现在文件中。

```
// 建立逻辑地址从 0x5000 开始、文件偏移从零开始的映像区域。
region.begin("image", 0x5000, 0);

emit.bytes(b"HDR0");
file_size_field:
emit.u32(0);
logical_size_field:
emit.u32(0);

// 中间三字节空隙会因后续的 0xee 而写入文件，末尾八字节只增加逻辑大小。
emit.u8(0xaa);
reserve(3);
emit.u8(0xee);
reserve(8);

defer {
    // 布局稳定后，分别回填文件大小与逻辑大小。
    store.u32(file_size_field, region_file_size(file_size_field));
    store.u32(logical_size_field, region_logical_size(logical_size_field));

    assert(load.u32(file_size_field) == 17);
    assert(load.u32(logical_size_field) == 25);
}
```

```
}
```

物理文件只有 17 字节：

```
48 44 52 30 11 00 00 00 19 00 00 00 aa 00 00 00 ee
```

中间的 3 字节预留因为后面还有已初始化数据而物理化。最后 8 字节预留只把逻辑大小增加到 25，不增加文件字节。

这适合零初始化存储等场景：文件记录已初始化前缀，剩余逻辑空间由加载器或消费程序提供。

Late Layout: 追加尾部或放置晚生成表

只有真实尾部必须追加，或表、字符串池、重定位记录这类内容必须等普通源码处理完后才能创建/放置时，才使用 `late_layout`。

```
// 文件开头先为最终大小写出一个四字节占位字段。
total_size:
emit.u32(0);
emit.bytes(b"DATA");

late_layout {
    // 普通源代码处理完成后，把真实尾部加入最终布局。
    emit.bytes(b"END!");
}

defer {
    // 收尾处理能够看到包含尾部在内的最终文件大小。
    store.u32(total_size, region_file_size(total_size));
    assert(load.u32(total_size) == 12);
}
```

输出：

```
0c 00 00 00 44 41 54 41 45 4e 44 21
```

追加的尾部会参与最终布局。`defer` 看到完整映像，并修补已经分配好的文件头字段。

只有这些追加字节确实必须晚于主源码创建时，才使用 `late layout`。普通源码顺序能够表达同样布局时，普通顺序更清楚。

如果 `late_layout` 里只调用 `emit.*`，它就是在默认输出尾部继续写；这也是最简单、最常见的尾部追加。若晚生成内容属于某个自定义表区或节数据区，应在块里显式 `region.begin(name, origin, file_offset)`，把后续字节放入那个真实区域。

这解决的是“布局尚未封存前创建真实字节”的问题，不是最终映像里的随机插入。已经物理化且位置固定的字段，用 `defer` 回填；缺少的变长表、字符串池、重定位记录等，必须在普通源码或 `late_layout` 中创建，然后再由 `defer` 回填指针、大小和校验。

构造类似 PE section 的自定义区域时要分清两层：

- `region.begin` 的名称只是 XIRASM 的布局区域名，不会自动生成 PE section 表项；
- 如果只是往当前文件尾部补一张表，直接尾部 `emit.*` 就够；
- 如果要把晚生成表放到数据区或某个中间文件偏移，必须显式切换到那块区域，并确保它不和已有文件范围冲突；
- 如果字段已经在头部预留，只需要最终值，使用 `defer`，不要用 `late_layout` 重新造空间；
- 标准 PE/COFF/ELF 优先使用格式接口，让接口维护 section/segment 表、raw pointer、virtual size 和回填字段。

断言确认文件正确

自定义写入器应该用断言保证输出可读：

- 签名和版本字段值符合预期
- 偏移指向区域内部
- 计数与写出的记录数一致
- 存的大小与区域最终信息匹配
- 校验和覆盖了正确的字节范围
- 逻辑大小和物理大小符合格式规则

`defer` 中的断言验证将要写出的精确字节，包括已编码指令和已解析 `fixup`。断言失败会停止汇编，避免产出看似有效但实际不合法的文件。

断言紧贴它保护的字段。大小回填和对应的断言通常在同一个 `defer` 块里。

自定义格式的组织顺序

小格式的源码组织顺序：

- 声明记录类型和常量
- 写出固定头字段（含占位符）
- 写出数据区
- 追加真正需要晚出的表或尾部
- 回填稳定字段并做断言

可复用 `include` 可以封装记录声明和写出过程。格式状态应通过参数和值显式传递，不要把互不相关的硬编码偏移散落在源码里。

格式变复杂后保持同样分离：

- 源码和过程决定有哪些记录
- 标签和区域描述布局
- `late_layout` 创建必须参与最终布局的字节
- `defer` 推导并验证稳定字段

何时使用格式接口

不要因为 `flat` 写出器能够表达字节，就手工拼标准可执行文件或目标文件格式。PE、COFF、ELF 还需要协调文件头、表、权限、导入、导出、重定位和加载器规则。

XIRASM 为这些任务提供面向用户的格式接口。语言指南只介绍它们的角色；普通接口的完整示例见《格式教程》，需要直接构造底层格式结构时再看《高级格式构造指南》。

[返回目录](#)

第 14 章：可执行文件与目标文件格式

PE、COFF、ELF 不止是数据加指令。它们还包含文件头、节区段/段表、权限、入口点、导入导出表、符号表、重定位表，由链接器和加载器按规则解析。

XIRASM 用格式库处理这些：

```
// 导入用于构造标准文件格式的统一接口。
import("format/format.inc");
```

格式接口让源码描述目标映像，而不是手写表计数、表行位置、文件偏移、虚拟地址和文件头字段。

本章只介绍语言层面的通用模式。完整的格式选项、导入、导出、重定位、共享库、目标文件和高级辅助函数，请看《格式教程》。需要直接控制格式字段和表项时，再阅读《高级格式构造指南》。

先声明映像再写内容

使用格式接口的程序，第一步是声明映像包含哪些 section 或 segment。每个描述符给出名称、用途和权限。

描述符列表是结构性的：

- 长度决定所需表的数量
- 顺序决定格式表的顺序
- 名称标识后续写入的内容块
- 用途选择格式特定的行为
- 权限成为 section/segment 属性

源文件不需要另外传入表计数或行号。增删描述符时，生成的格式结构会随之更新。

完整示例：ELF64 程序

以下示例创建 x86-64 ELF 可执行文件，含一个可加载、可读、可执行的 segment：

```
// 导入格式接口，并选择 64 位 x86 指令编码。
import("format/format.inc");
x86.use64();

// 声明 ELF64 可执行映像及其唯一的可装载代码段。
let image: map = format_elf64(
  format_elf_exec,
  list.of(
    format_segment(
      ".text",
      format_load | format_readable | format_executable
    )
  )
)
format_begin(image);

// 在声明的 .text 段中写入程序入口代码。
format_segment_begin(image, ".text");
start:
  // 调用 Linux 退出系统调用，并把退出状态设为零。
  mov eax, 60
  xor edi, edi
  syscall
format_segment_end(image, ".text");

// 绑定入口标号，随后完成映像中的文件头和表项。
format_entry_mut(image, start)
format_finish(image);
```

在 x86-64 Linux 下，这个程序以状态码 0 退出。

五步走：

- `format_elf64` 创建声明式 ELF64 计划。
- `format_segment` 声明一个 segment 及其权限。
- `format_begin` 写出计划决定的格式结构。
- `format_segment_begin` 和 `format_segment_end` 包住这个 segment 的实际指令和数据。
- `format_entry_mut` 绑定入口标号，`format_finish` 完成映像。

源文件从不提供程序头计数、表行、文件偏移、加载地址或入口点字段偏移。格式接口会从计划和已完成的 segment 布局推导这些值。

Sections 与 Segments

普通生命周期沿用所选格式的术语：

- PE 映像和目标文件用 section
- ELF 可执行文件和共享库用 segment
- ELF 目标文件用 section，不涉及运行时装载

对应的生命周期调用：

```
format_section_begin(image, name)
format_section_end(image, name)
```

```
format_segment_begin(image, name)
format_segment_end(image, name)
```

只打开计划中声明过的描述符，并按声明时的用途使用每个名称。这样格式接口才能把已写出的字节、逻辑大小、权限和生成的表关联到正确的格式条目。

格式族

格式库支持以下映像类：

映像类	常规构造函数
PE32 / PE64 映像	<code>format_pe32</code> 、 <code>format_pe64</code>
COFF32 / COFF64 目标文件	<code>format_coff32</code> 、 <code>format_coff64</code>
ELF32 / ELF64 可执行文件	<code>format_elf32</code> 、 <code>format_elf64</code>
ELF32 / ELF64 目标文件	<code>format_elfobj32</code> 、 <code>format_elfobj64</code>
ELF64 共享库	<code>format_elf64_so</code>

构造函数选项区分可执行文件与 DLL、控制台/GUI 子系统、位置无关执行、NX 策略、地址随机化等映像属性。Section/segment 描述符携带内容用途，以及读、写、执行或可丢弃等权限。

完整的选项和描述符列表属于参考材料，本语言指南不重复列出。

入口点绑定

可执行文件应在源码定义完入口代码后绑定入口标号：

```
format_entry_mut(image, start)
format_finish(image);
```

`format_entry_mut` 会直接更新第一个参数传入的 `let` 绑定；这里不需要维护多个不可变中间副本。`format_finish` 随后验证可执行文件具备所需入口信息。

目标文件和部分库形式不需要入口点。按所选格式的生命周期来，不要加无意义的入口标签。

生成的格式内容

部分格式内容来自更高层的声明，不是作为普通 payload 字节手写出来的：

- 导入导出表
- 符号表和字符串表
- 重定位记录
- 基址重定位段
- 动态元数据
- 资源目录

这些功能的格式接口 API 接受名称、标号、权限和重定位类型。section 编号、符号索引、表偏移、行号和计数由接口内部推导。

不要用硬编码的表位置替代这些声明。格式接口的价值正在于让格式簿记与实际布局保持一致。

常规接口与高级接口

普通程序应从 `format/format.inc` 开始。它是稳定的用户层，负责协调格式属性、描述符、命名内容块、入口和生成表。

各格式特有的 include 文件提供更低层次的构建块。有些暴露特定定位宽的便捷封装，有些暴露直接的文件头、section、segment 或表辅助函数。它们适合实现新的格式接口，或表达普通层无法覆盖的专用布局。

只是生成常见的多 section 可执行文件、DLL、目标文件、共享库、导入表、导出表或重定位表时，不需要切到底层 include。普通接口支持目标映像时，应优先使用普通接口。

不要随意混用抽象层次。底层辅助函数可能要求调用者自己管理计数、行号、偏移或格式不变式，而这些通常由普通接口维护。

第 11 到 13 章的 `region.begin`、虚拟区域、`late_layout` 和 `defer` 是底层布局工具。它们能表达“字节在文件哪里、逻辑地址是多少、最终字段何时回填”，但不会自动生成 PE/COFF/ELF 的 section、segment、符号表或重定位表记录。需要这些格式记录时，用 `format_section_begin`、`format_segment_begin` 和对应格式 API；只有在实现新格式接口或普通接口覆盖不了的专用布局时，才直接组合底层区域和收尾阶段。

后续阅读

《格式教程》提供可直接使用的模板、参数说明和 API 摘要：

- PE32/PE64 可执行文件和 DLL
- COFF32/COFF64 目标文件
- ELF32/ELF64 可执行文件和位置无关可执行文件
- ELF32/ELF64 目标文件
- ELF64 共享库
- 多 section 和多 segment
- 已初始化和未初始化数据
- 导入、导出、符号、重定位

- 普通格式接口与高级底层辅助函数的界限

已知所需格式系列时直接查 API Reference。

下一章讲诊断和源文件组织惯例。

[返回目录](#)

第 15 章：诊断与实用惯例

当源码把自己的假设说清楚时，汇编错误最容易修。XIRASM 提供了用于信息输出、非致命警告、主动失败和不变式检查的诊断工具。

源码组织也遵循同一个原则：命名清晰、坐标明确、辅助函数小而直接，并选择正确的抽象层级，二进制布局才容易审查，也更容易修改。

阅读诊断信息

诊断会指出源文件位置、严重级别和说明：

```
source.asm:12:5: error: header size must be four
```

该位置指向报告或触发问题的语句。解析、Meta 求值、指令编码、布局、fixup 解析和收尾处理都使用同一套面向源码的位置报告模型。

先看第一个错误。后面的失败可能只是早先的无效声明、缺失标号或被拒绝指令连带出来的结果。

信息与警告

print 输出信息，**warn** 输出非致命警告：

```
// 输出当前位置，帮助用户了解本次汇编采用的映像起点。
print("image origin", here());
// 输出仍然有效，但使用默认对齐值时给出明确提醒。
warn("using default alignment", 16);
// 确认继续汇编所需的配置条件成立。
assert(true, "configuration must be valid");
// 诊断信息不会改变输出内容，这里实际写出一个字节。
emit.u8(0x7d);
```

汇编照常成功，输出：

```
7d
```

消息后的参数值也会打出：

```
note: image origin 0
warning: using default alignment 16
```

有意保留的汇编时信息才用 **print**，不要把永久调试噪声留在源码里。输出仍然有效、但所选配置值得注意时用 **warn**。

警告不会改变输出，也不会让汇编失败。如果某个条件会产生非法文件或不受支持的程序，应改用错误。

主动报错

err 打出致命错误并终止：

```
if target.bits != 64 {
    err("this source requires a 64-bit target", target.bits);
}
```

额外参数会格式化在消息之后。传入导致配置无效的值，可以帮助使用者修正源码。

消息应说明违反的要求：

```
err("section alignment must be a nonzero power of two", alignment);
```

当源码能够说出预期属性时，不要只写 **invalid value** 这种空泛消息。

断言

assert 是编码不变式最直接的方式：

```
// 定义由两个 16 位字段组成的紧凑文件头布局。
packed struct Header {
    kind: u16
    size: u16
}

// 文件头大小发生变化时立即停止汇编。
assert(sizeof(Header) == 4, "Header must remain four bytes");

// 写出文件头及其后紧接的三个字节数据。
emit.struct(Header {
    kind: 1,
    size: 3,
});
emit.bytes(b"ABC");
```

断言通过时不产生输出。完成后的字节是：

```
01 00 03 00 41 42 43
```

条件为假时，汇编停止，提供的消息会成为错误文本。

普通源码处理阶段已经知道的事实，直接用普通 **assert**：

- 类型大小和字段偏移
- 选项组合
- 列表和 map 内容
- 目标需求
- 常量范围
- 声明值之间的关系

需要稳定最终映像才能确认的事实，放在 **defer** 的 **assert** 里：

- 最终文件和逻辑大小
- 已解析的偏移和地址
- 回填的文件头字段
- 校验和
- 生成表的内容
- 指令 fixup 产生的字节

把断言放在输入事实变稳定的同一阶段，可以避免无意依赖临时布局值。

保护目标相关源码

把目标需求写在依赖它的代码附近：

```
// 明确选择 64 位 x86 指令编码。
x86.use64();

// 不满足位数要求时，在汇编期间直接拒绝该目标平台。
if target.bits != 64 {
    err("this routine requires x86-64");
}

entry:
// 例程只为 x86-64 生成清零和返回指令。
xor eax, eax
ret
```

输出：

```
31 c0 c3
```

目标条件在汇编时求值；它们选择或拒绝源码，不会生成运行时检查。

显式选择模式也让可复用源码更容易审查。读者不应靠猜来判断某段指令是写给 x86-16、x86-32、x86-64、RISC-V 32 还是 RISC-V 64。

用名字替代魔数

带有格式、协议或布局含义的数值应取名字：

```
const header_size: u32 = 16
const page_alignment: u64 = 0x1000
const record_kind_code: u16 = 3
```

算法内部的数值操作数可以保持局部。偏移、标志、结构大小、格式值和对齐策略通常应该命名。

当数值代表封闭集合中的一个选项时，用命名常量。多个字段组成一条记录时，用 `struct`。源码构造动态计划时，用 `map` 或 `list`。

在名称中写清坐标

二进制写入器经常同时处理多套坐标。名称应体现坐标含义：

```
header_foa
text_rva
entry_address
payload_file_size
payload_logical_size
```

当 `offset` 可能指文件偏移、区域相对偏移、虚拟地址或结构字段偏移时，不要使用这种泛称。

辅助函数参数也一样。接受 `logical_address` 和 `file_offset` 的函数，比接受 `start` 和 `position` 的函数更不容易被误用。

推导布局事实

- 逻辑距离 → 标签相减
- 结构体布局 → `sizeof`、`offset_of`
- 当前物理光标 → `file_offset()`
- 稳定区域事实 → `defer` 里查
- 表计数和顺序 → 格式描述符列表决定
- 生成索引 → 符号和重定位声明决定

只有外部格式规范要求精确常量时，才硬编码偏移。即使硬编码，也要给常量取描述性名字，并尽可能断言周围布局。

保持收尾处理聚焦

普通源文件按普通顺序写出。

只有真实字节或区域必须在主源码之后追加时，才用 `late_layout`。只有需要检查或修补稳定映像时，才用 `defer`。

不要只是为了逃避源码组织，就把普通写出挪到后期阶段。一个明确的“文件头、payload、尾部、收尾处理”序列，比一张延迟副作用网络更容易理解。

让每个 `defer` 聚焦一组相关字段。一个回填并验证文件头的块，比一个到处修改无关区域的块更容易审查。

按职责分模块

实用的项目布局通常按职责拆分：

- 共享常量和数据类型
- 可重用的写出过程
- 目标相关指令例程
- 格式或容器构造
- 数据输入
- 顶层源文件（选配置、拼最终映像）

只需初始化一次的模块用 `import`。确实需要重复文本求值时才用 `include`。

公共辅助函数应明确说明输入。可以直接传 `plan`、`list`、`map`、`label` 或 `option` 时，不要依赖隐藏的可变状态。

选择正确的格式层

标准可执行文件和目标文件从 `format/format.inc` 开始：

```
// 导入用于构造标准文件格式的常规接口。  
import("format/format.inc");
```

普通格式接口负责描述符计数、表顺序、生成索引、偏移和常见格式不变式。

只有在实现新的格式接口，或普通接口无法表达所需布局时，才使用底层格式 `include`。不要仅仅因为高级 API 暴露了更多数字就切换层级；那些数字通常是普通层应继续维护的簿记。

项目专用文件应使用第 13 章的 `flat` 与自定义二进制技术，不要强行塞进操作系统格式。

在输出边界检查

在 XIRASM 把输出交给外部系统的边界处验证：

- 写自定义文件前断言最终结构
- 显式声明可执行文件的权限和重定位策略
- 写出前验证导入数据
- 尽早拒绝不支持的目标组合
- 入口点和导出符号保持命名
- 每个公共 `include` 保留一个可运行的小例子

验证应描述约定，而不是重复实现细节。断言表计数与声明列表一致很有用；断言每一步内部算术则更难维护。

找到合适的文档

本文档（语言指南）管：

- 编译时源文件和 ISA 文本如何协作
- 值、控制流、函数、集合、`token` 的行为
- 标签、数据、区域、虚拟输出、收尾处理如何组织文件
- `flat` 输出和格式库的选择

API Reference 管：

- 精确的函数签名
- 接受的值类型
- 常量和选项标志
- 行为和错误约束

《格式教程》管：

- 完整的 PE、COFF、ELF 程序
- 可执行文件、DLL、目标文件、PIE、共享库
- 导入、导出、符号、资源、重定位
- 多 section 和多 segment
- 普通格式接口的参数、调用顺序和常见错误

这些文档把概念解释、查询材料和格式专用流程分开，各自保持可读。

最终检查清单

源文件完成前确认：

- ☐ 目标和指令模式显式指定
- ☐ 常量和值类型有描述性名字
- ☐ 标签替代硬编码地址
- ☐ 外部依赖的结构体大小已断言
- ☐ import 和 include 用对了模型
- ☐ 预留的尾部和物理间隙是故意的
- ☐ late_layout 只创建真正晚出的字节
- ☐ 收尾处理修改已有存储并验证稳定事实
- ☐ 标准格式用格式库，除非有高级需求
- ☐ 致命条件给出可操作的消息
- ☐ 最终输出有可复现的汇编或执行验证

[返回目录](#)